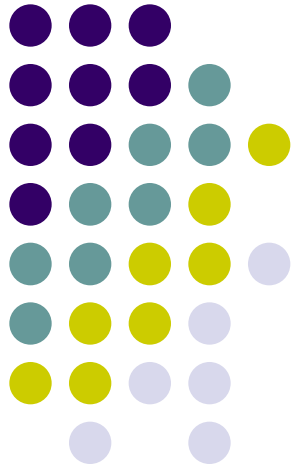
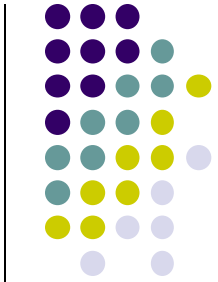
Stack & Queue Implementations



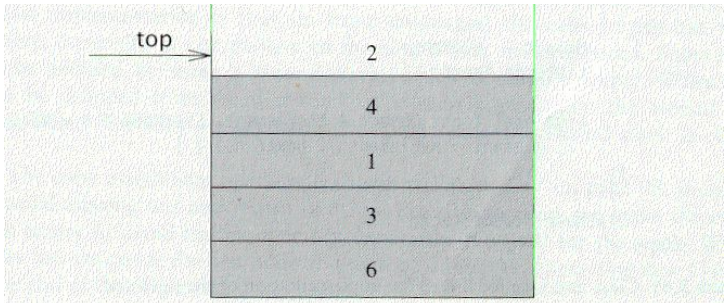
Chapter Contents

- A Sequential Implementation
- A Linked Implementation
- A Vector-Based Implementation



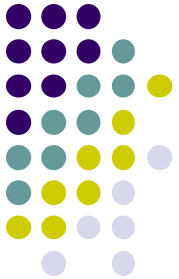
The Stack ADT

- A stack is a list with the restriction
  - that insertions and deletions can only be performed at the *top* of the list



  - The other end is called bottom
- Fundamental operations:
  - Push: Equivalent to an insert
  - Pop: Deletes the most recently inserted element
  - Top: Examines the most recently inserted element

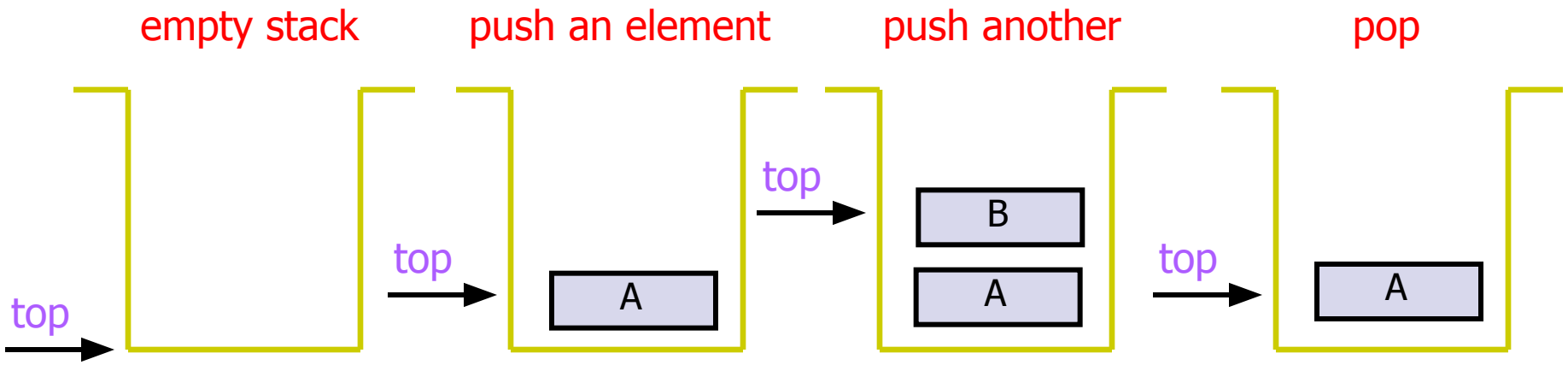
Stack ADT



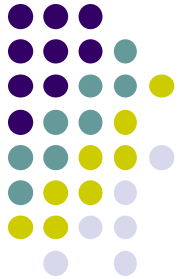
- Stacks are less flexible
  - ✔ but are more efficient and easy to implement
- Stacks are known as LIFO (Last In, First Out) lists.
  - The last element inserted will be the first to be retrieved

Push and Pop

- Primary operations: Push and Pop
- Push
  - Add an element to the top of the stack
- Pop
  - Remove the element at the top of the stack

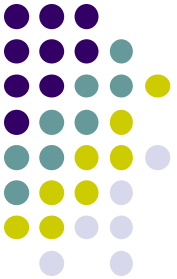


Implementation of Stacks



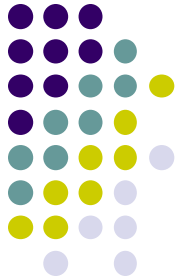
- Any list implementation could be used to implement a stack
  - Arrays (static: the size of stack is given initially)
  - Linked lists (dynamic: never become full)
- We will explore implementations based on array and linked list
- Let's see how to use an array to implement a stack first

Sequential Implementation



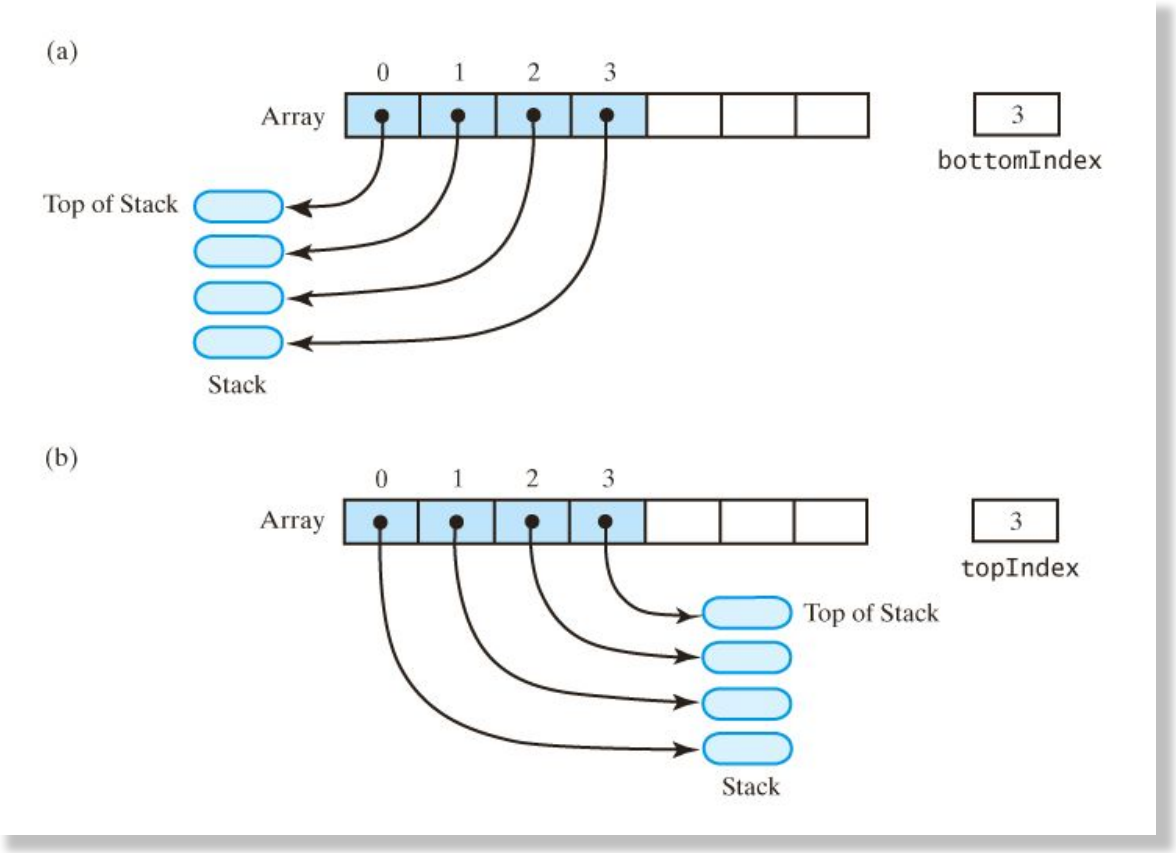
- Need to declare an array size ahead of time
- Associated with each stack is TopOfStack
  - for an empty stack, set TopOfStack to -1
- Push
  - (1) Increment TopOfStack by 1.
  - (2) Set Stack[TopOfStack] = X
- Pop
  - (1) Set return value to Stack[TopOfStack]
  - (2) Decrement TopOfStack by 1
- These operations are performed in very fast constant time

Sequential Implementation



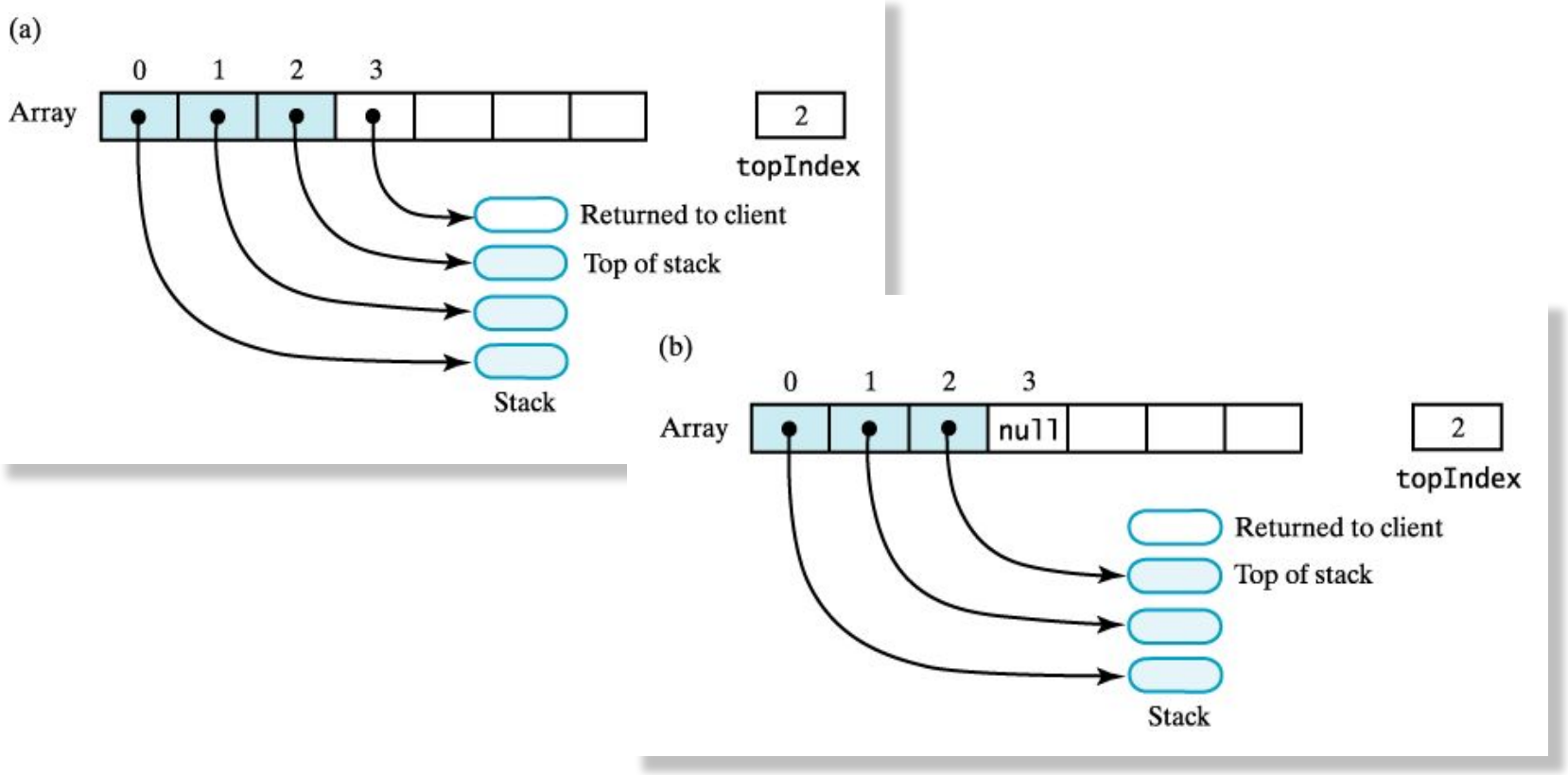
- When using an array to implement a stack
  - The array's first element should represent the bottom of the stack
  - The last occupied location in the array represents the stack's top
- This avoids shifting of elements of the array when we push or remove elements from stack

Sequential Implementation of Stack



An array that implements a stack; its first location references (a) the top of the stack; (b) the bottom of the stack

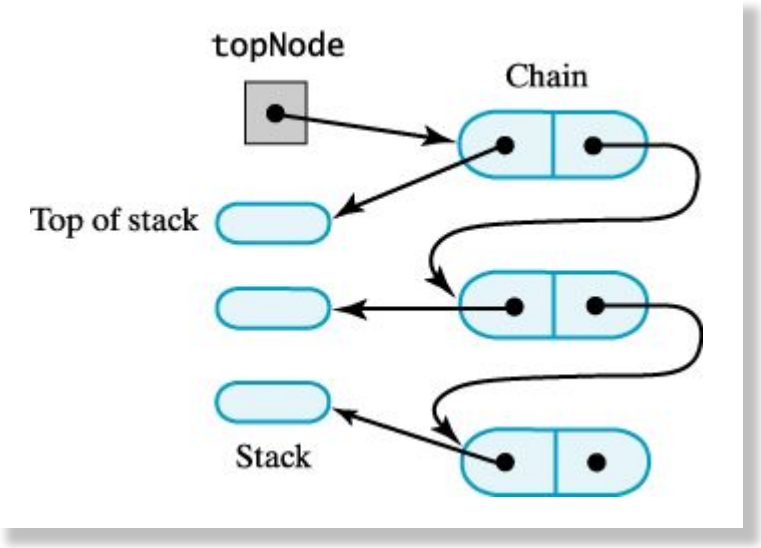
Sequential Implementation : Pop



Another array-based stack after top removed by (a) decrementing `topIndex`; (b) setting `stack[topIndex]=null` and then decrementing `topIndex` to be safe

A Linked Implementation

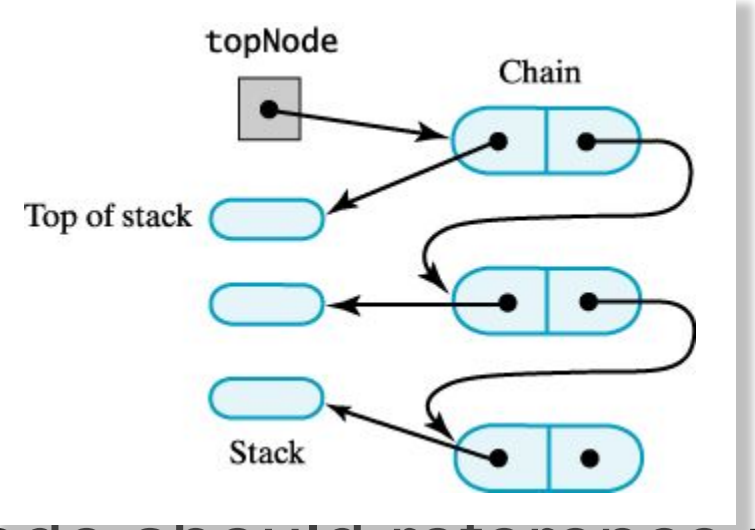
- When using a chain of linked nodes to implement a stack with each node reference one entry in stack



A chain of linked nodes that implements a stack.

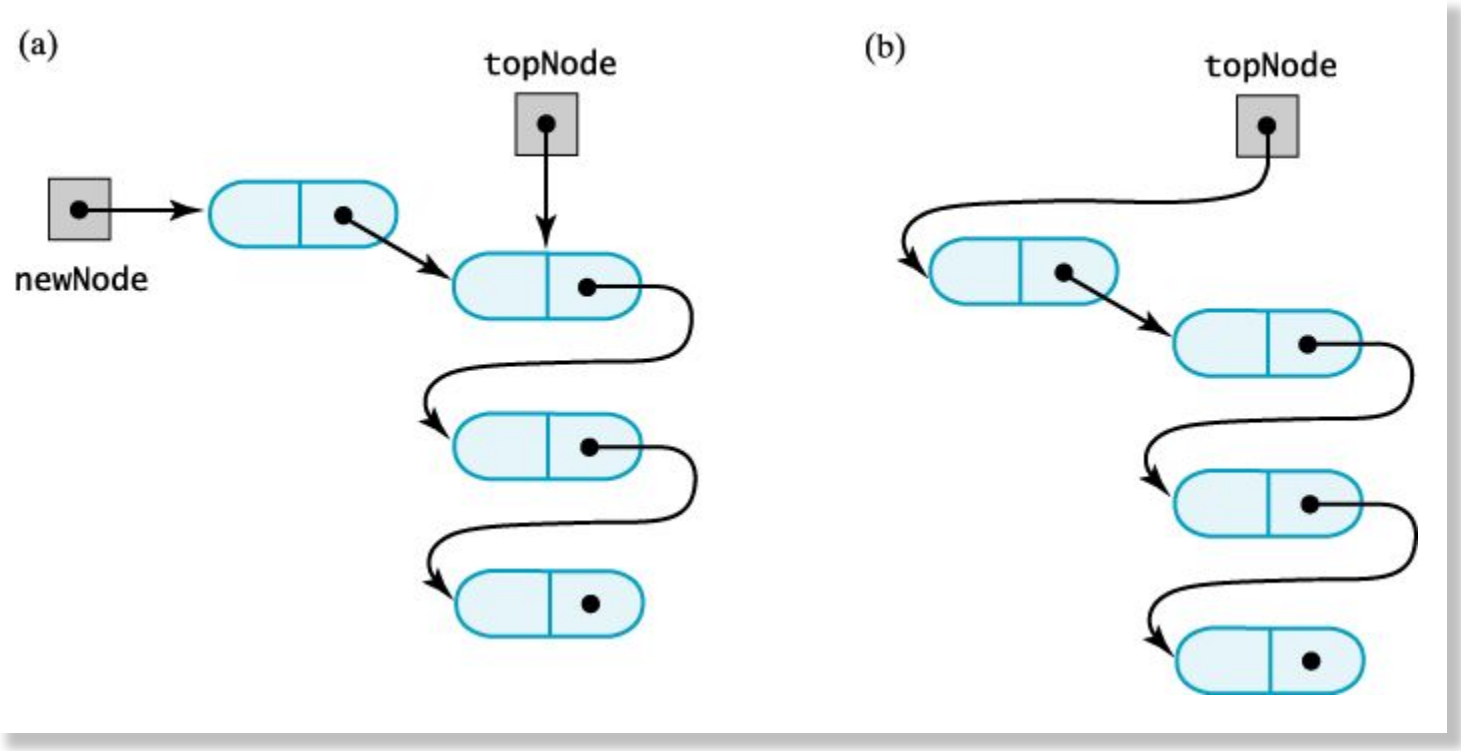
A Linked Implementation

- When a chain has only a head reference, we can add, remove or retrieve its first node faster than any other node.



- The first node should reference the stack's top

A Linked Implementation: Push or Add to the top

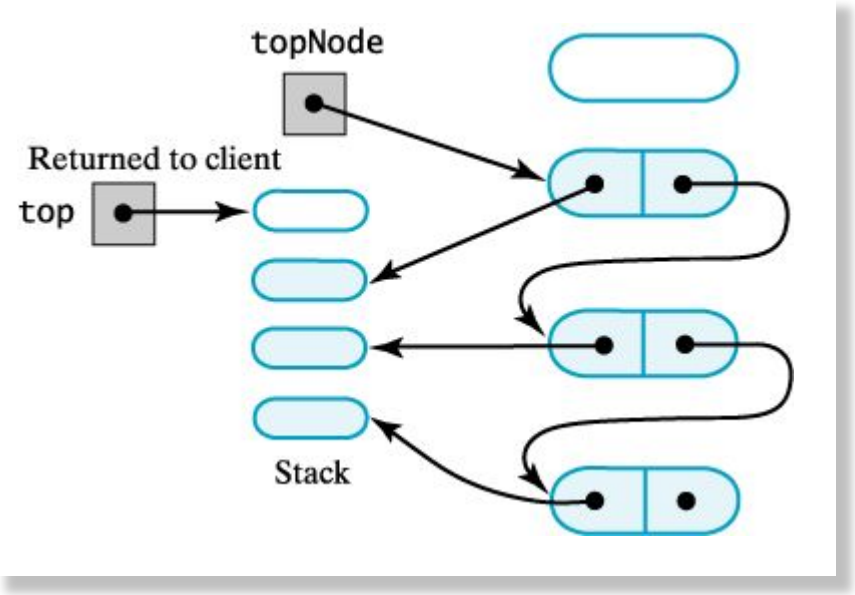


Push a new node to the stack

(a) allocate memory for newnode

(b) topNode references newNode.

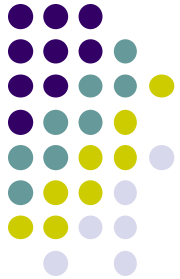
A Linked Implementation: Pop or Remove



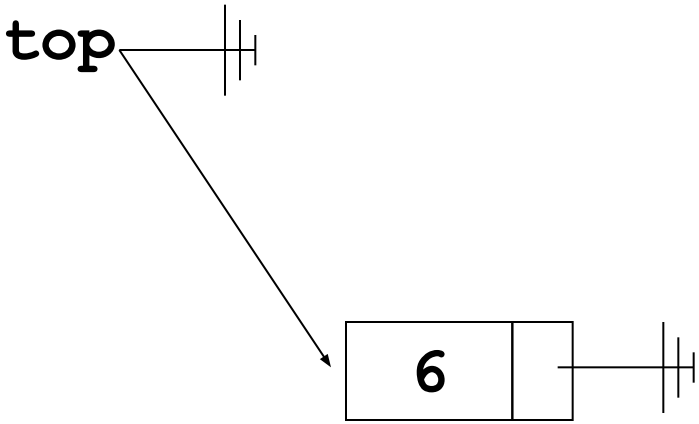
The stack after the first node in the chain is deleted.

```
public T pop()
{
 T top = peek();
 If (topNode != null)
  topNode = topNode.getNextNode();
 return top;
}
```

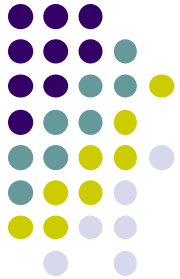
List Stack Example



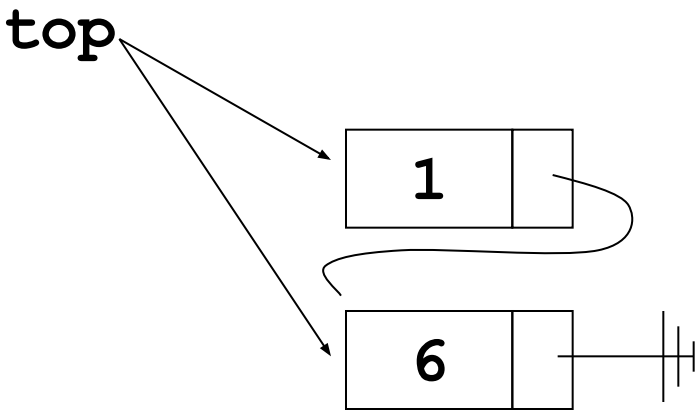
```
push(6);
```



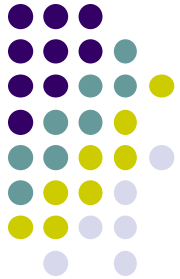
List Stack Example



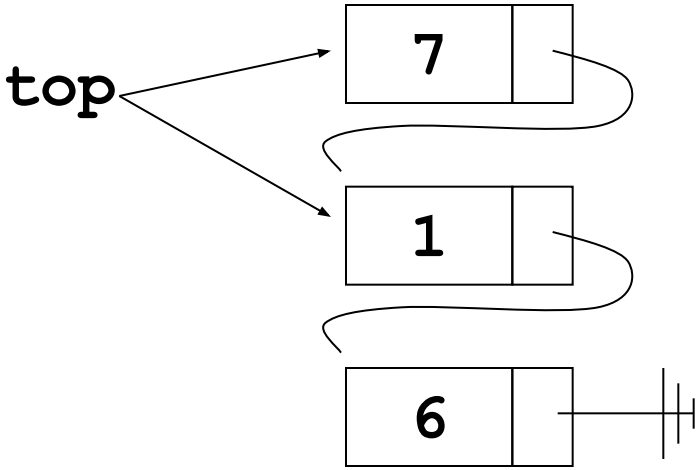
```
push(6);
push(1);
```



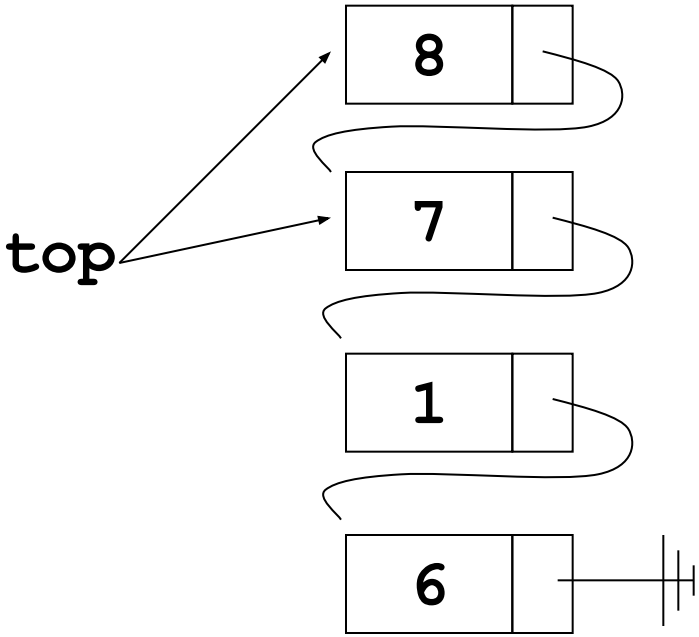
List Stack Example



```
push(6);
push(1);
push(7);
```

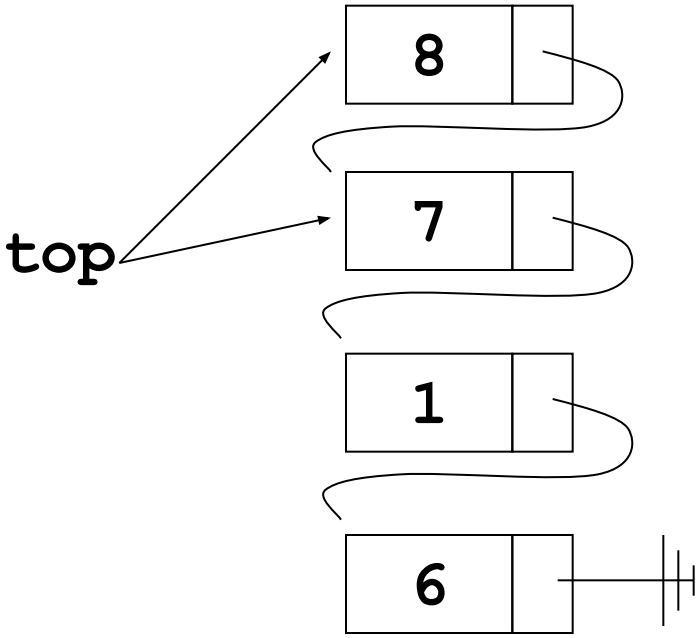


List Stack Example



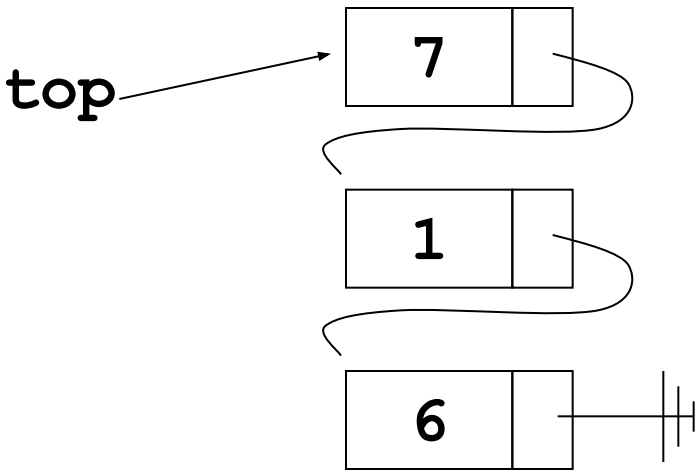

```
push(6);
push(1);
push(7);
push(8);
```

List Stack Example



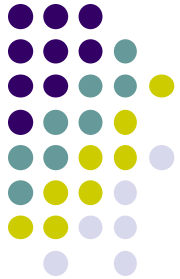

```
push(6);
push(1);
push(7);
push(8);
pop();
```

List Stack Example

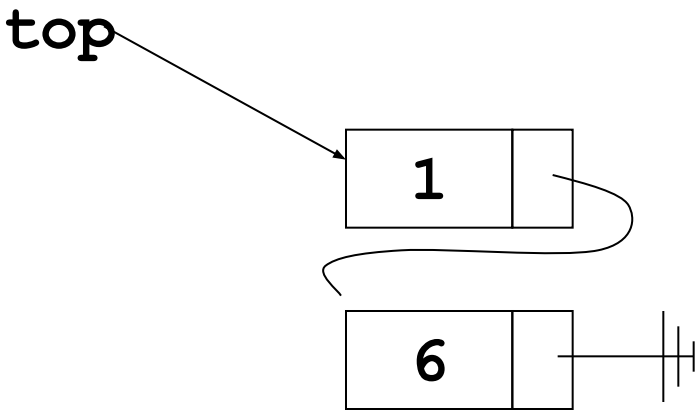


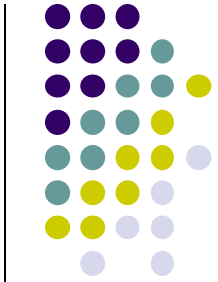

```
push(6);
push(1);
push(7);
push(8);
pop();
```

List Stack Example



```
push(6);
push(1);
push(7);
push(8);
pop();
Pop();
```





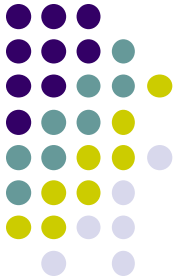
Applications of Stacks

- Call stack (recursion).
- Searching networks, traversing trees (keeping a track where we are).

Examples:

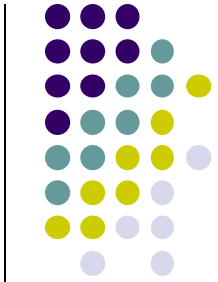
- Checking balanced expressions
- Recognizing palindromes
- Evaluating algebraic expressions

Evaluating Postfix Expressions



- A postfix (reverse Polish logic) calculator
  - Requires you to enter postfix expressions
    - Example: 2 3 4 + *
  - When an operand is entered, the calculator
    - Pushes it onto a stack
  - When an operator is entered, the calculator
    - Applies it to the top two operands of the stack
    - Pops the operands from the stack
    - Pushes the result of the operation on the stack

Algorithm for Infix to Postfix



1) Examine the next element in the input.
2) If it is operand, output it.
3) If it is opening parenthesis, push it on stack.
4) If it is an operator, then
   i) If stack is empty, push operator on stack.
   ii) If the top of stack is opening parenthesis, push operator on stack
   iii) If it has higher priority than the top of stack, push operator on stack.
   iv) Else pop the operator from the stack and output it, repeat step 4
5) If it is a closing parenthesis, pop operators from stack and output them until an opening parenthesis is encountered. pop and discard the opening parenthesis.
6) If there is more input go to step 1
7) If there is no more input, pop the remaining operators to output.

Suppose we want to convert 2*3/(2-1)+5*3 into Postfix form,



Expression	Stack	Output
2	Empty	2
*	*	2
3	*	23
/	/	23*
(	/(	23*
2	/(	23*2
-	/(-	23*2
1	/(-	23*21
)	/	23*21-
+	+	23*21-/
5	+	23*21-/5
*	+*	23*21-/53
3	+*	23*21-/53
	Empty	23*21-/53*+

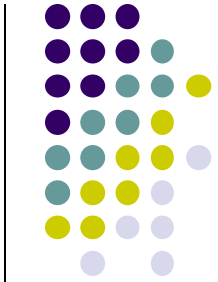
So, the Postfix Expression is 23*21-/53*+

Example

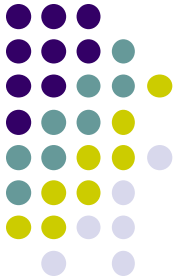
- (5 + 6) * 9 +10

will be

- 5 6 + 9 * 10 +

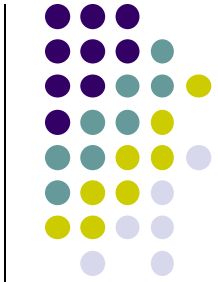


Evaluation a postfix expression



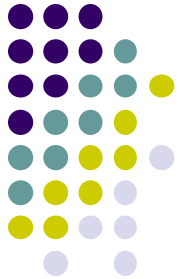
- Each operator in a postfix string refers to the previous two operands in the string.
- Suppose that each time we read an operand we **push** it into a stack. When we reach an operator, its operands will then be top two elements on the stack
- We can then **pop** these two elements, perform the indicated operation on them, and **push** the result on the stack.
- So that it will be available for use as an operand of the next operator.

Evaluating Postfix Notation



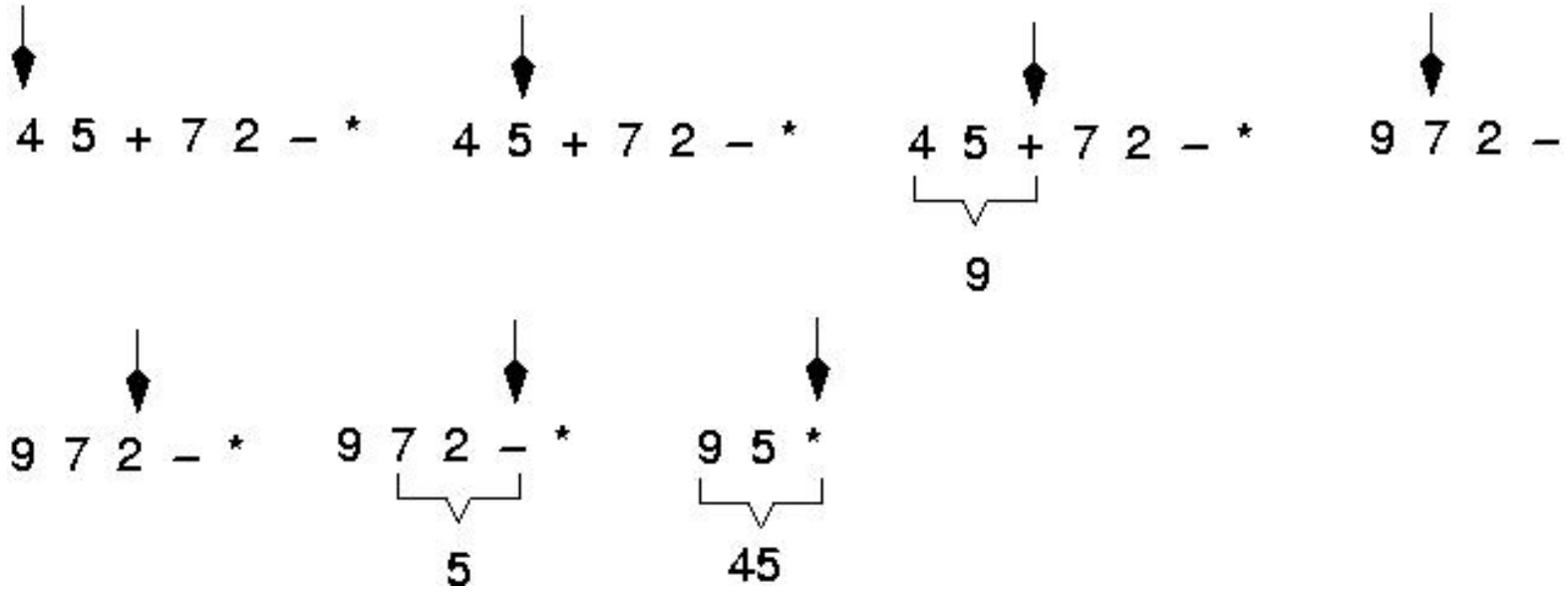
- Use a stack to evaluate an expression in postfix notation.
- The postfix expression to be evaluated is scanned from left to right.
- Variables or constants are pushed onto the stack.
- When an operator is encountered, the indicated action is performed using the top elements of the stack, and the result replaces the operands on the stack.

Evaluating a postfix expression

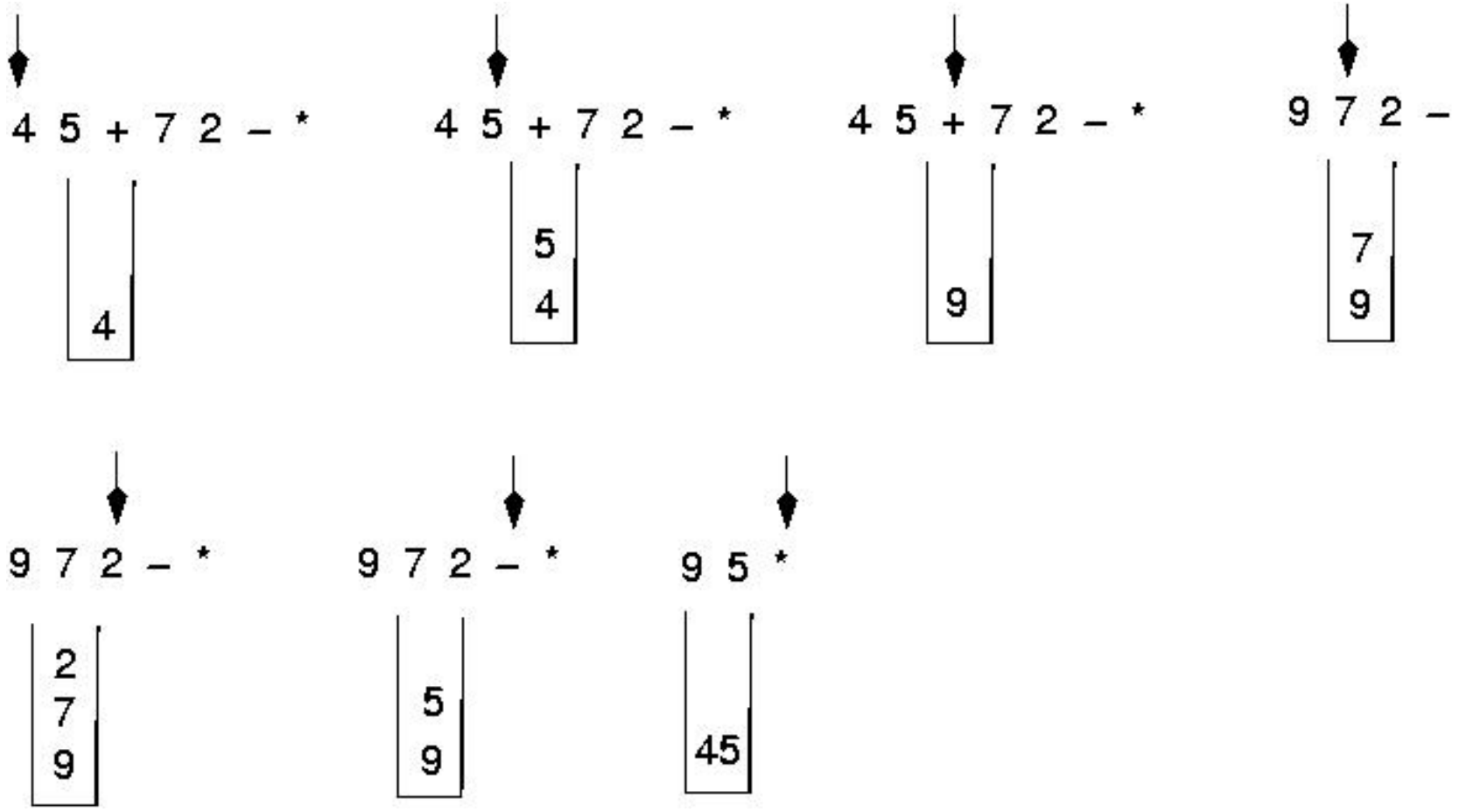


- Initialise an empty stack
- While token remain in the input stream
  - Read next token
  - If token is a number, push it into the stack
  - Else, if token is an operator, pop top two tokens off the stack,apply the operator, and push the answer back into the stack
- Pop the answer off the stack.

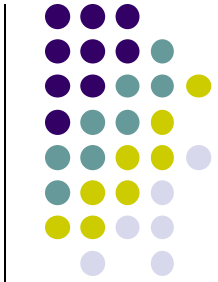
Example: postfix expressions (cont.)



Postfix expressions: Algorithm using stacks (cont.)

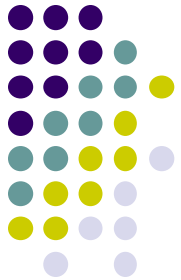


Algorithm for evaluating a postfix expression (Cond.)



```
WHILE more input items exist
{
    If symb is an operand
     then push (opndstk,symb)
    else  //symbol is an operator
    {
      Opnd1=pop(opndstk);
      Opnd2=pop(opndnstk);
      Value = result of applying symb to opnd1 & opnd2
      Push(opndstk,value);
    }       //End of else
} // end while
Result = pop (opndstk);
```

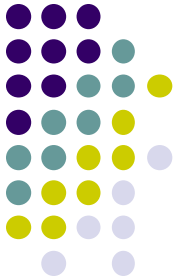
Question : Evaluate the following expression in postfix : 623+-382/+*2^3+



Final answer is

- 49
- 51
- 52
- 7
- None of these

Evaluate- 623+-382/+*2^3+

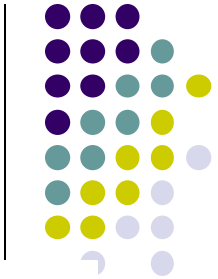


Symbol	opnd1	opnd2	value	opndstk
6				6
2				6,2
3				6,2,3
+	2	3	5	6,5
-	6	5	1	1
3	6	5	1	1,3

Evaluate- 623+-382/+*2^3+

Symbol	opnd1	opnd2	value	opndstk
8	6	5	1	1,3,8
2	6	5	1	1,3,8,2
/	8	2	4	1,3,4
+	3	4	7	1,7
*	1	7	7	7
2	1	7	7	7,2
^	7	2	49	49
3	7	2	49	49,3
+	49	3	52	52

Evaluating Postfix Expressions



Key entered	Calculator action		Stack (bottom to top)
2	push 2		2
3	push 3		2 3
4	push 4		2 3 4
+	operand2 = pop stack	(4)	2 3
	operand1 = pop stack	(3)	2
	result = operand1 + operand2	(7)	2
	push result		2 7
*	operand2 = pop stack	(7)	2
	operand1 = pop stack	(2)	
	result = operand1 * operand2	(14)	
	push result		14

The action of a postfix calculator when evaluating the expression 2 * (3 + 4)

Evaluating Postfix Expressions



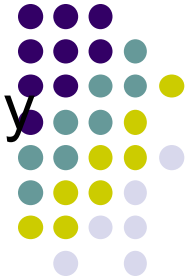
- Pseudo code:

```
int evaluate(String expression)
{
  Stack stack=new Stack(); // creaty empty stack
  while (true) {
    String c=expression.getNextItem();
    if (c==ENDOFLINE)
      return stack.pop();

    if (c is operand)
      stack.push(c);
    else { // operation
      int operand2=stack.pop();
      int operand1=stack.pop();
      stack.push(execute(c,operand1,operand2));
    }
  }
}
```

Algorithm 3.6 Evaluation of a Postfix Expression

```
evaluationofpostfix(s, postfix)
 1. Set i = 0, RES=0.0
 2. While (i < number_of_characters_in_postfix)
       If postfix[i] is a whitespace or comma
          Set i = i + 1 and continue
       If postfix[i] is an operand, push it onto the stack
       If postfix[i] is an operator, follow these steps:
          i.   Pop the top element from stack and store it in
                                                      operand2
          ii.  Pop the next top element from stack and store it in
                                                      operand1
          iii. Evaluate operand2 op operand1 and store the result
                                 in RES (op is the current operator)
          iv.  Push RES back to stack
       End If
       Set i = i + 1
    End While
 3. Pop the top element and store it in RES
 4. Return RES
 5. End
```



- Data abstraction, or abstract data types, is a programming methodology where one defines not only the data structure to be used, but the processes to manipulate the structure
  - like process abstraction, ADTs can be supported directly by programming languages
- To support it, there needs to be mechanisms for
  - defining data structures
  - encapsulation of data structures and their routines to manipulate the structures into one unit
    - by placing all definitions in one unit, it can be compiled at one time
  - information hiding to protect the data structure from outside interference or manipulation
    - the data structure should only be accessible from code encapsulated with it so that the structure is hidden and protected from the outside
    - objects are one way to implement ADTs, but because objects have additional properties

RECURSION



- When a function definition includes a call to itself, it is referred to as a **recursive function** and the process is known as **recursion** or **circular definition.**
- In each recursive call, the current values of the parameters, local variables and the return address where the control has to return from the call are required to be stored. For storing all these values, a stack is maintained.
- When a recursive function is called for the first time, a space is set aside in the memory to execute this call and the function body is executed. Then a second call to the function is made; again a space is set for this call and so on. These memory areas for each function call are arranged in the stack. Each time the function is called, its memory area is placed on the top of the stack and is removed when the execution of the call is completed

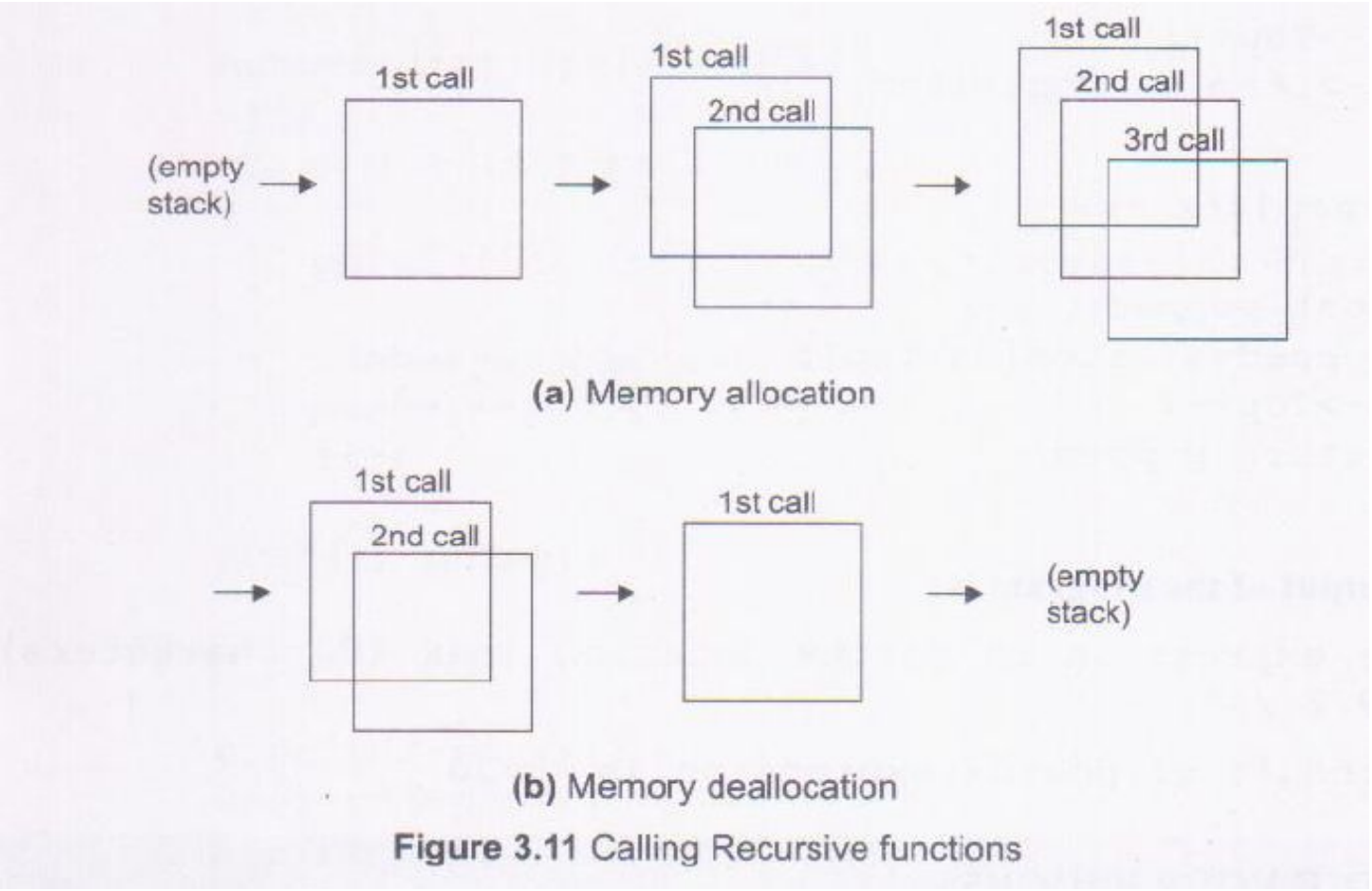
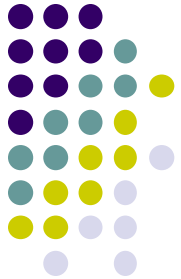


Figure 3.11 Calling Recursive functions

APPLICATION OF RECURSION:

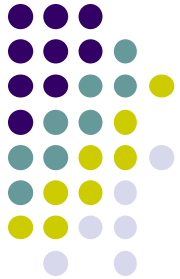
1.computing factorial of a given number.
2.finding the Fibonacci series.
3.determining the GCD (Greatest Common Divisor) of two positive numbers.
4.computing binary equivalent of a decimal number.
5.towers of Hanoi problem.

Queues



- A queue is a data structure that only allows items to be inserted at the end and removed from the front
- “Queue” is the British word for a line (or line-up)
- Queues are **FIFO** (First In First Out) data structures – “fair” data structures

Definitions



- A queue is a linear abstract data type such that insertions are made at one end, called the **rear**, and removals are made at the other end, called the **front.**
- Queues are sometimes called **FIFOs**: first-in first-out.

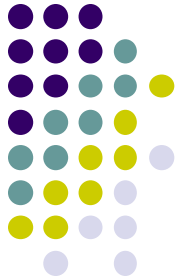
enqueue() → Queue → dequeue()

The two basic operations are:

- enqueue: adds an element to the rear of the queue.
- dequeue: removes and returns the element at the front of the queue.

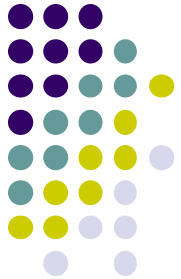
→Software queues are similar to physical ones: queuing at the supermarket, at the bank, at cinemas, etc.

Applications



- Shared resources management (system programming):
  - Access to the processor;
  - Access to the peripherals such as disks and printers.
- Application programs:
  - Simulations;
  - Generating sequences of increasing length over a finite size alphabet;

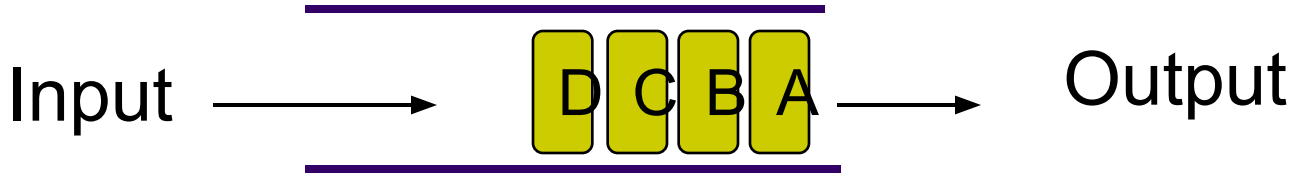
Implementations



- Just like stacks, there are two families of implementations.
  - Array-based;
  - Linked-structure.

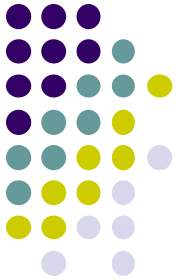
Queue

- Queue: *First In First Out (FIFO)*



- Toll Station
  - Car comes, pays, leaves
- Check-out in Big Y market
  - Customer comes, checks out and leaves
- More examples: Printer, Office Hours, …

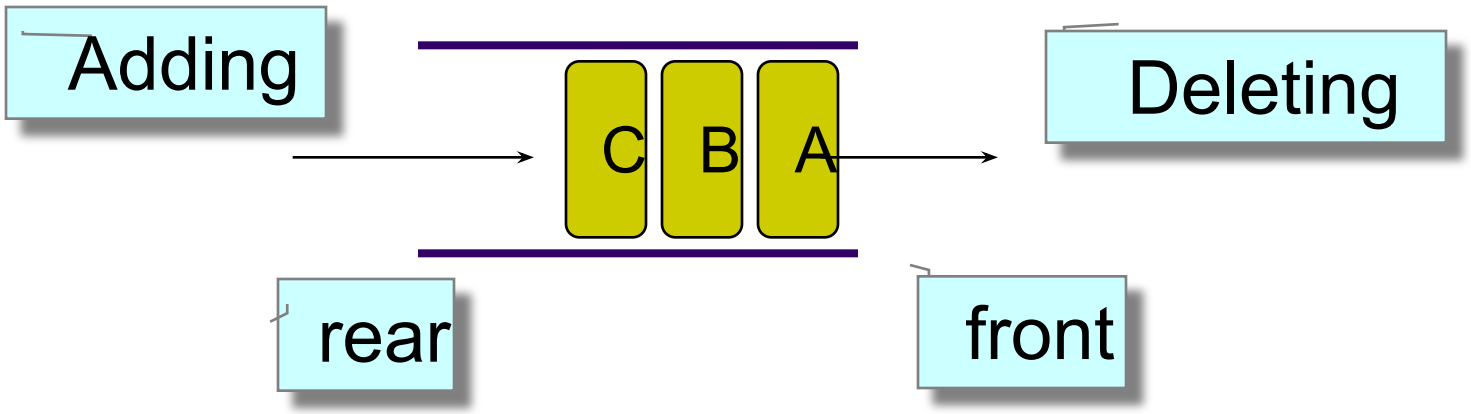
More Examples of Queue



- In our daily life
  - Airport Security Check
  - Cinema Ticket Office
  - Bank, ATM
  - Anything else ?

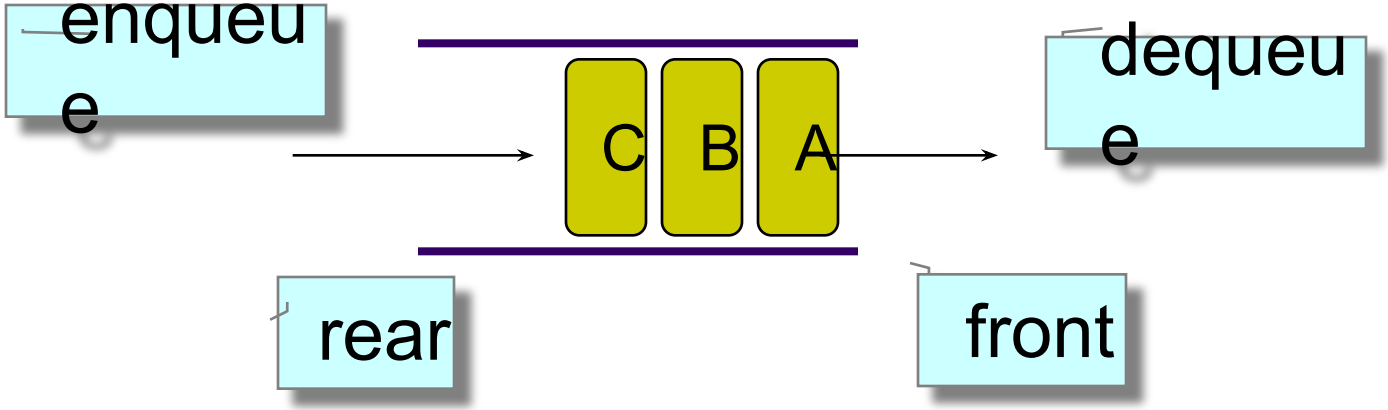
What Is Queue

- Queue is an abstract data type
- Adding an entry at the rear
- Deleting an entry at the front

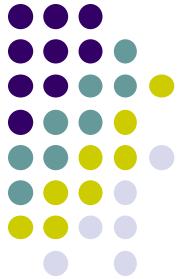


Abstract Data Types

- Queue
  - Operating on both ends called **Doubled ended** Queue
  - Operations: EnQueue(in), DeQueue(out)



Queue

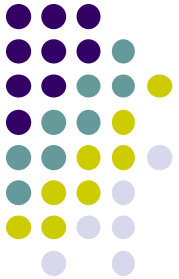


Queue is FIFO (First-In First-Out)

A queue is open at two ends. You can only add entry (enqueue) at the rear , and delete entry (dequeue) at the front.

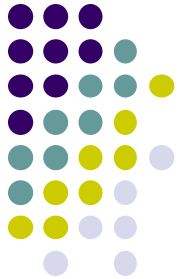
Note that you cannot add/extract entry in the middle of the queue.

Applications of Queue



- Printing Job Management
- Packet Forwarding in Routers
- Message queue in Windows
- I/O buffer

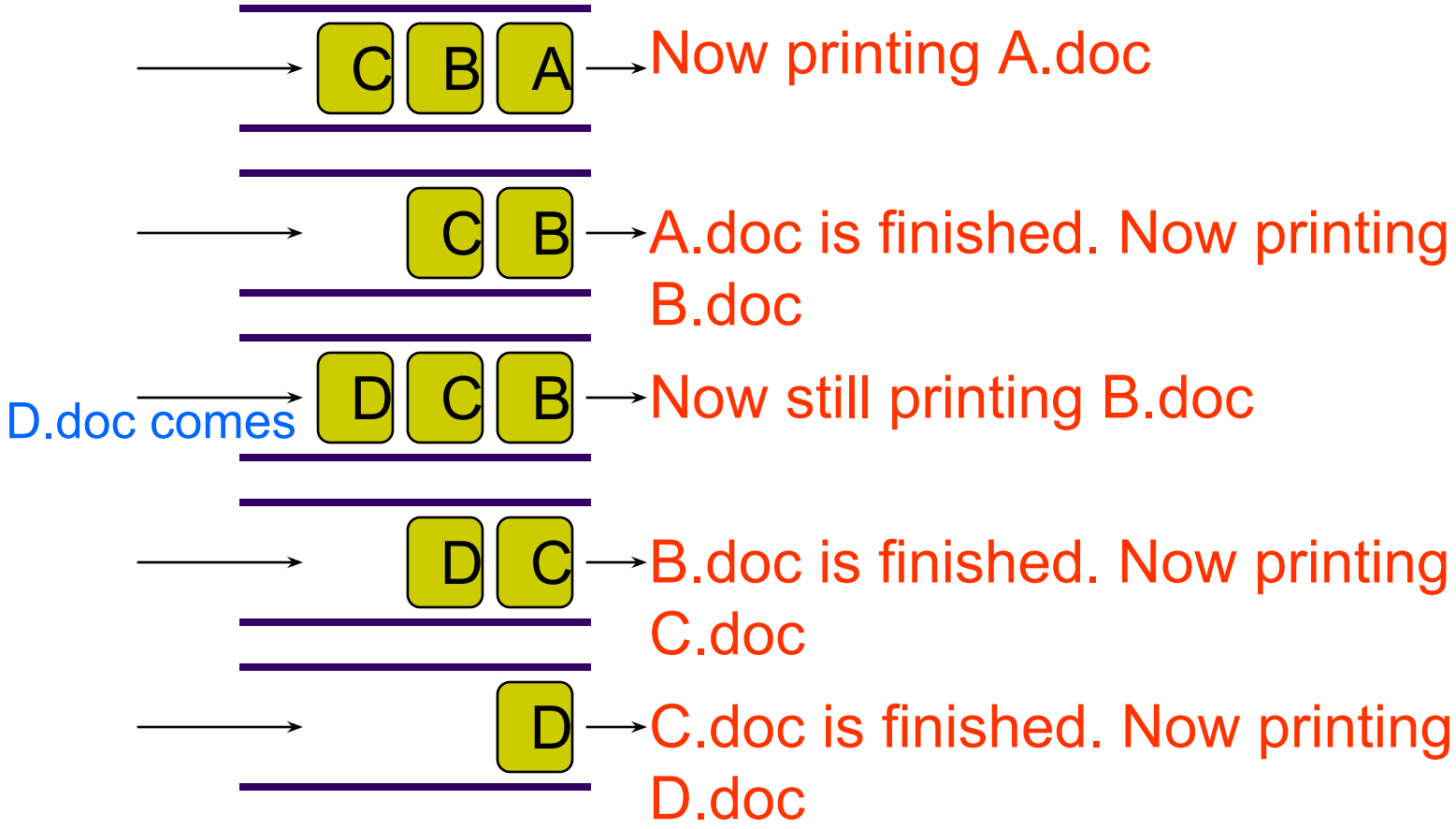
Printing Job Management



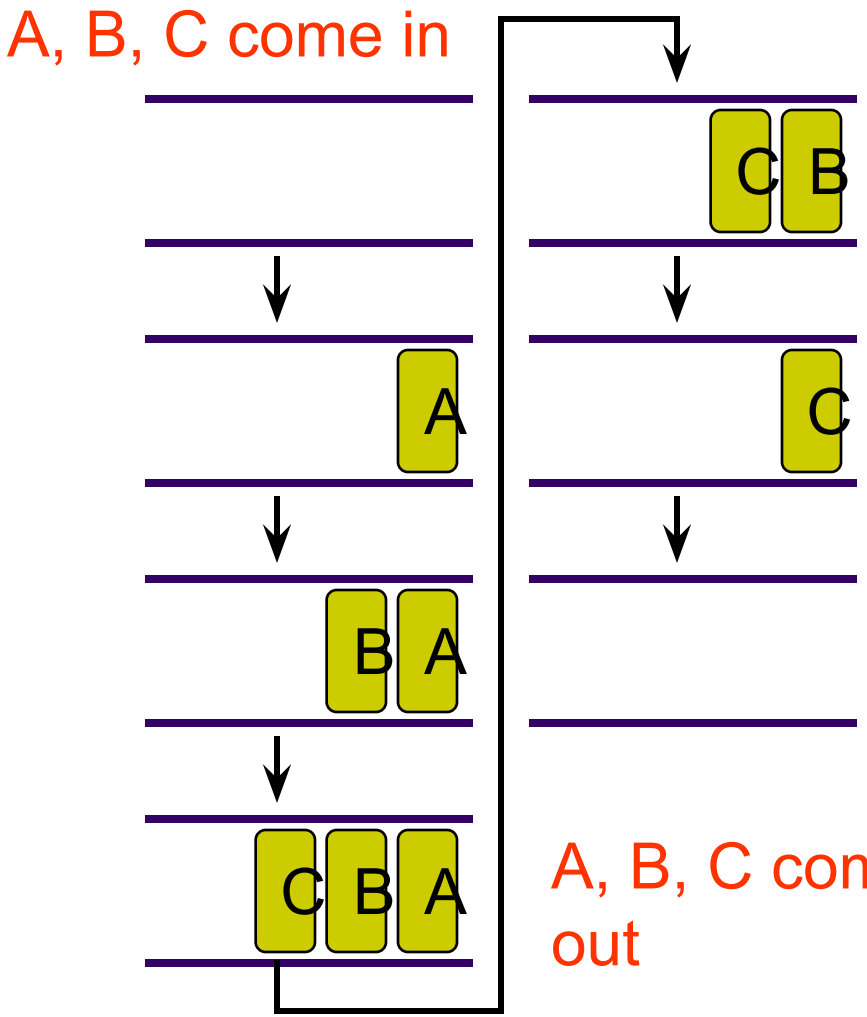
- Many users send their printing jobs to a public printer
- Printer will put them into a queue according to the arrival time and print the jobs one by one
- These printing documents are A.doc, B.doc, C.doc and D.doc

Printing Queue

- A.doc B.doc C.doc arrive to printer.



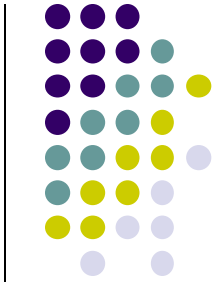
First-in First-out (FIFO)



The first one enqueued is the first one dequeued. (FIFO)

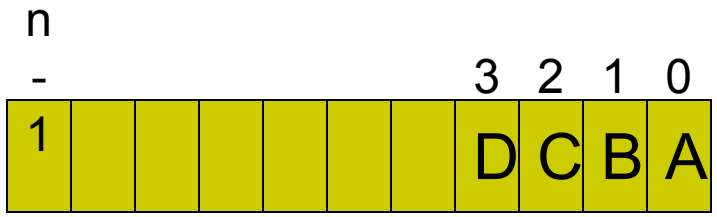
When we enqueue entries in the queue and then dequeue them one by one, we will get the items in the same order.

Abstract Data Type

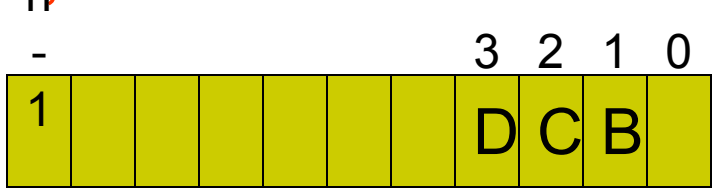


- Same as Stack, we use Object data type instead of int or double or String or other data type
- Use an array in queue, which stores all items come in.
  - Object Queue[ ];
- Other implementations are possible

Array Implementation of Queue

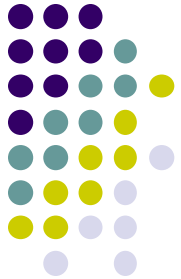


After A leaves,



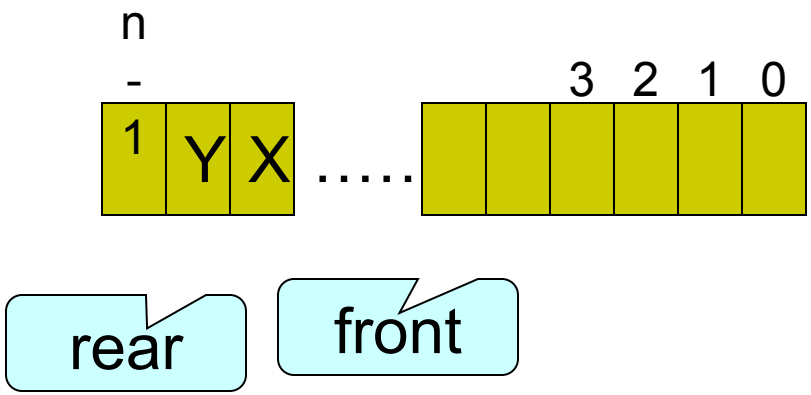
Operations

- enqueue
  - add a new item at the rear
- dequeue
  - remove a item from the front
- isEmpty
  - check whether the queue is empty or not
- isFull
  - check whether the queue is full or not
- size
  - return the number of items in the queue
- peek
  - return the front item



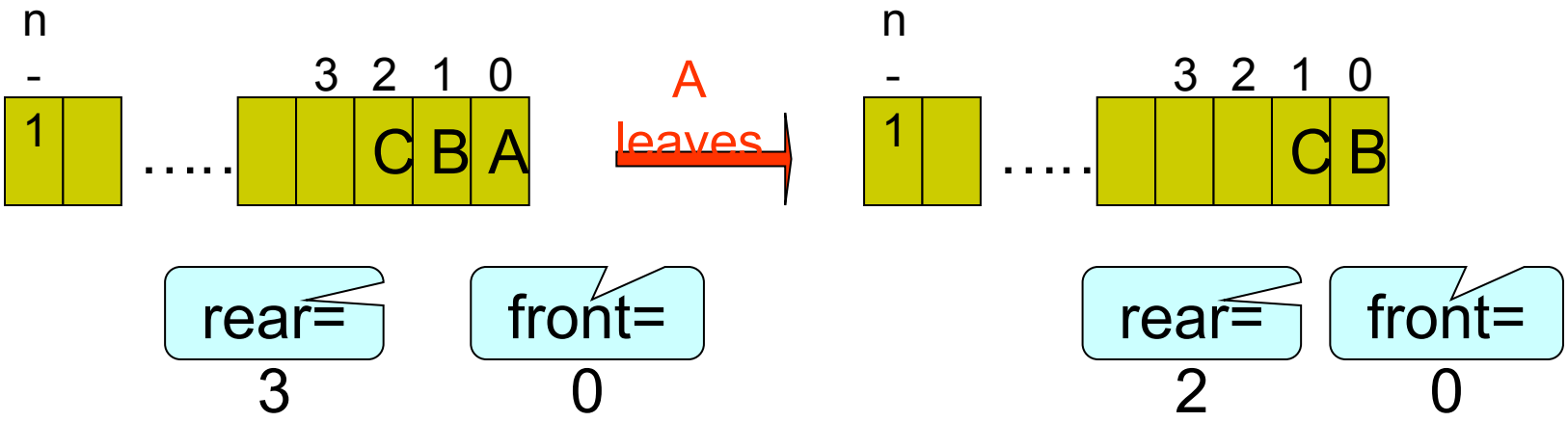
Problem

- An array has limited size, once rear is at the end of this array, and there is new item coming in, what can we do?



Two Solutions

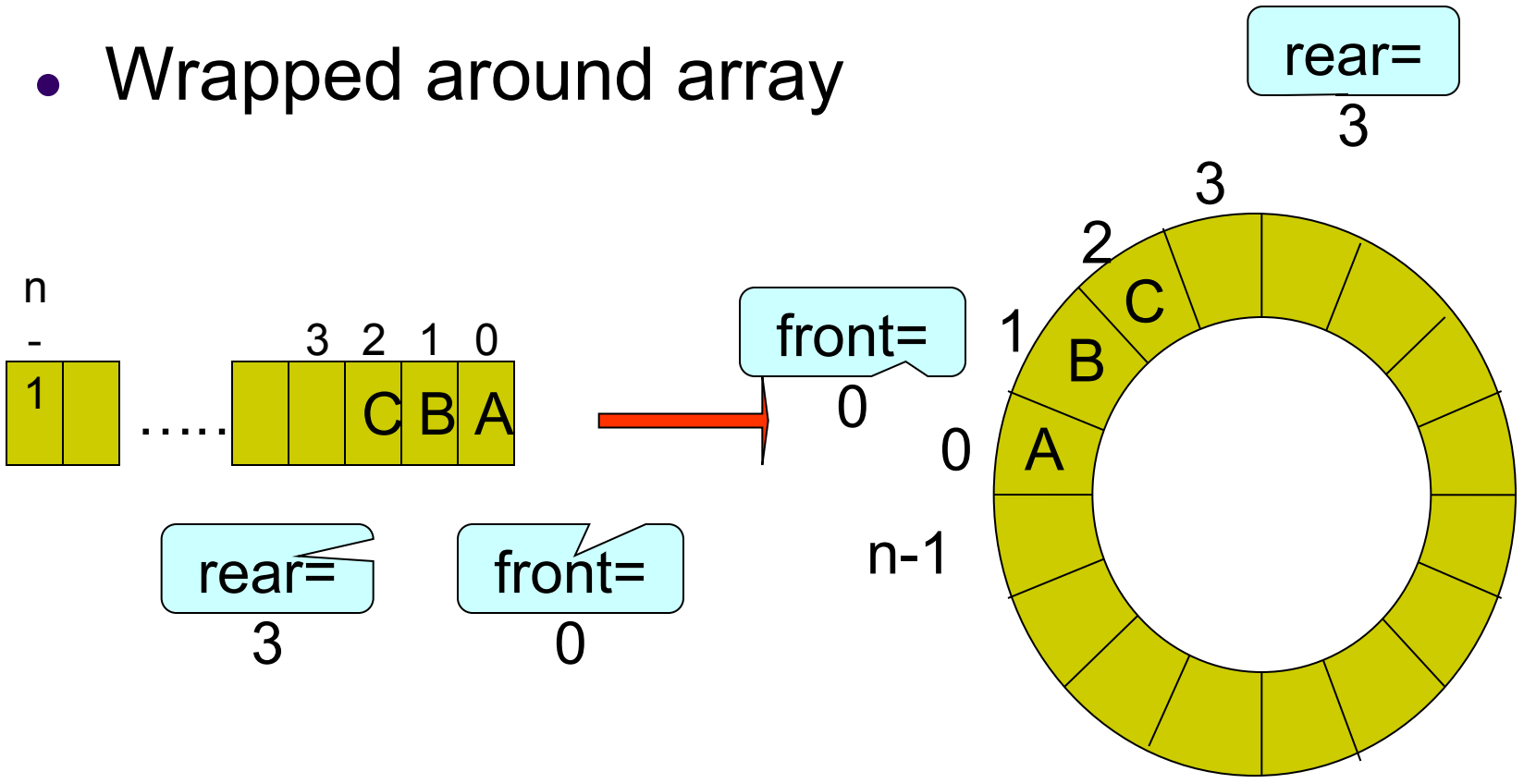
- Shifting all items to front in the array when dequeue operation. (Too Costly…)



- Wrapped around array ---- Circular Array

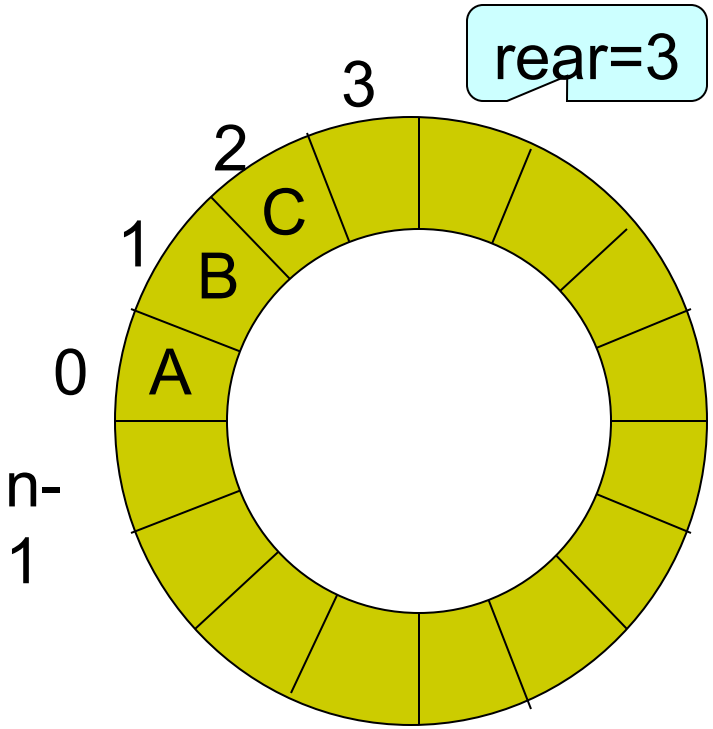
Circular Queue

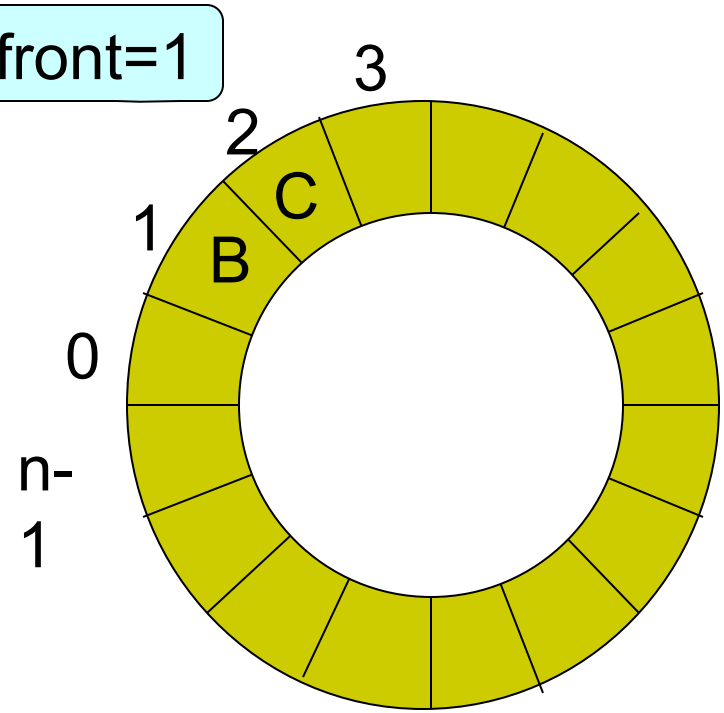
- Wrapped around array



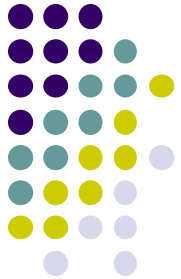
EnQueue & DeQueue In Circular Queue

- EnQueue
  - rear = (rear + 1) MOD n
- DeQueue
  - front = (front + 1) MOD n



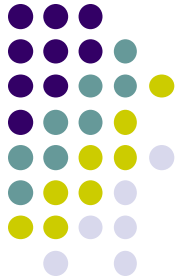


Empty/Full In Circular Queue



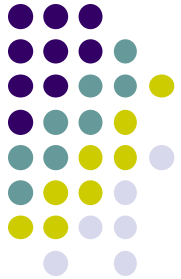
- When rear equals front, Queue is empty
- When (rear + 1) MOD n equals front, Queue is full
- Circular array with capacity *n* at most can hold *n-1* items.

What Can You Use a Queue For?



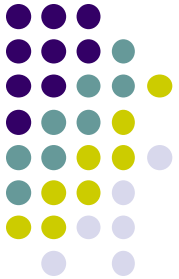
- Processing inputs and outputs to screen (console)
- Server requests
  - Instant messaging servers queue up incoming messages
  - Database requests
- Print queues
  - One printer for dozens of computers
- Operating systems use queues to schedule CPU jobs
- Simulations

Queue Operations



- A queue should implement (at least) these operations:
  - **enqueue** – insert an item at the back of the queue
  - **dequeue** – remove an item from the front
  - **peek** – return the item at the front of the queue without removing it
- Like stacks it is assumed that these operations will be implemented efficiently
  - That is, in constant time

Sequential Implementation of Queue



- First consider using an array as the underlying structure for a queue, one plan would be to
  - Make the back of the queue the current size of the queue (i.e., the number of elements stored)
  - Make the front of the queue index 0
  - Inserting an item can be performed in constant time
  - But removing an item would require shifting all elements in the queue to the left which is **too slow**!
- **Therefore we need to find another way**

Sequential Implementation of Queue

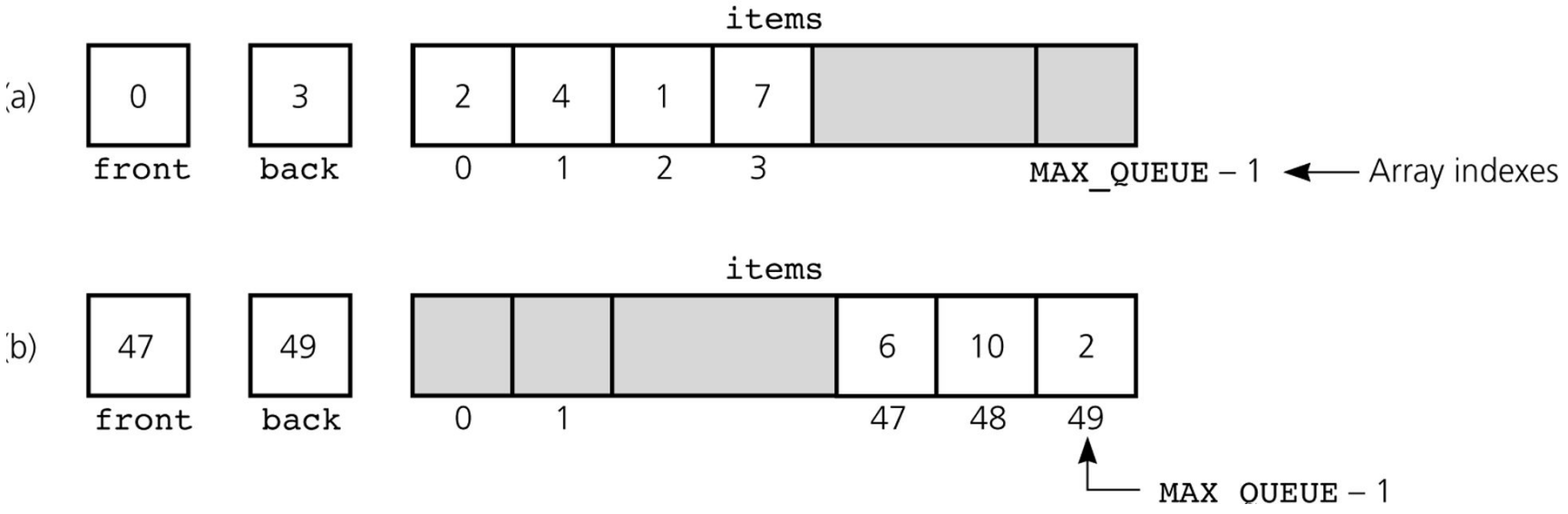
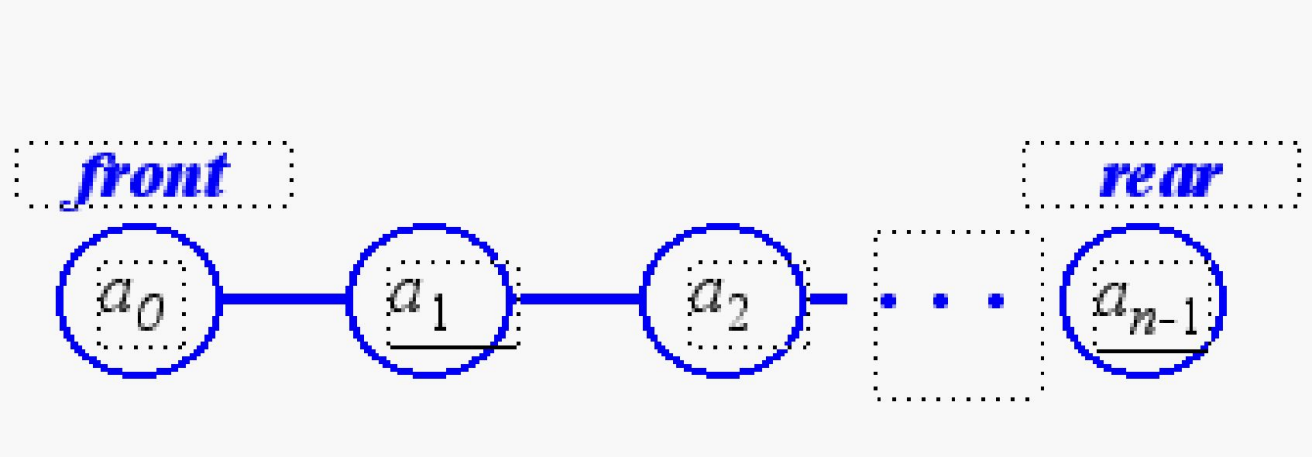


Figure 8-8

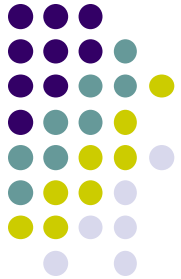
a) A naive array-based implementation of a queue; b) rightward drift can cause the queue to appear full

Queues

- A queue differs from a stack in that its insertion and removal routines follows the first-in-first-out (FIFO) principle.
- Elements may be inserted at any time, but only the element which has been in the queue the longest may be removed.
- Elements are inserted at the rear (enqueued) and removed from the front (dequeued)



The Queue Abstract Data Type



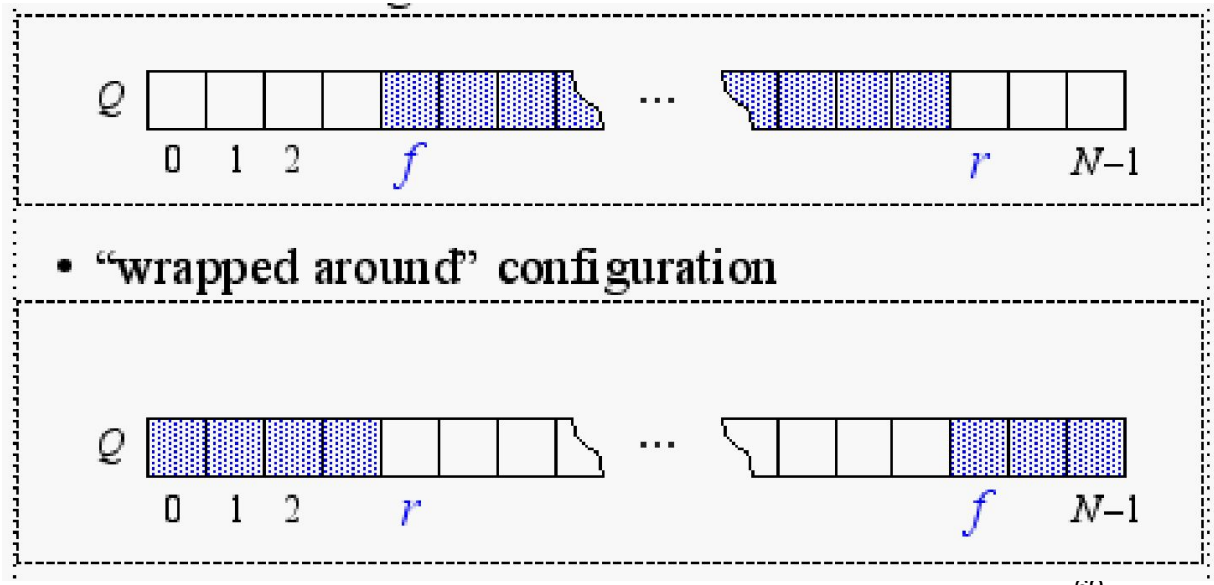
- The queue has two fundamental methods:
  - **enqueue(o):** Insert object o at the rear of the queue
  - **dequeue():** Remove the object from the front of the queue and return it; an error occurs if the queue is empty

- These support methods should also be defined:
  - **size():** Return the number of objects in the queue
  - **isEmpty():** Return a boolean value that indicates whether the queue is empty
  - **front():** Return, but do not remove, the front object in the queue; an error occurs if the queue is empty

Sequential Implementation of Queue

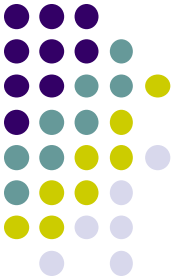


- Create a queue using an array in a circular fashion
- A maximum size N is specified, e.g. N = 1,000.
- The queue consists of an N-element array Q and two integer variables:
  - *f*, index of the front element
  - *r*, index of the element after the rear one
- "normal configuration"



- **Questions:**
  - ***What does f=r mean?***
  - ***How do we compute the number of elements in the queue from f and r?***

Sequential Implementation of Queue



Pseudo-Code (cont'd.)

```
Algorithm size():
    return (N - f + r) mod N

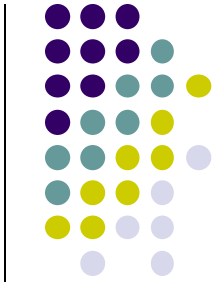
Algorithm isEmpty():
    return (f = r)

Algorithm front():
    if isEmpty() then
        throw a
QueueEmptyException
    return Q[f]
```

```
Algorithm dequeue():
    if isEmpty() then
        throw a
QueueEmptyException
    temp ← Q[f]
    Q[f] ← null
    f ← (f + 1) mod N
    return temp

Algorithm enqueue(o):
    if size = N - 1 then
        throw a
QueueFullException
    Q[r] ← o
```

Linked list



- Def: A **sequence** composed of many **Nodes**, each Node is composed of two parts: **Data**, **Link** (or Pointer; point to another node).
- A **node** in a linked list is a structure that has at least two fields:
  - **Data**:
    - The data part holds the useful information, the data to be processed.
  - **Link**:
    - Used to chain the data together. It contains a pointer that identifies the next element in the list.
- A **pointer variable** identifies **the first element in the list**. The name of the list is the same as the name of this pointer variable.

Note: different representations of a node

- If the node have its own name:



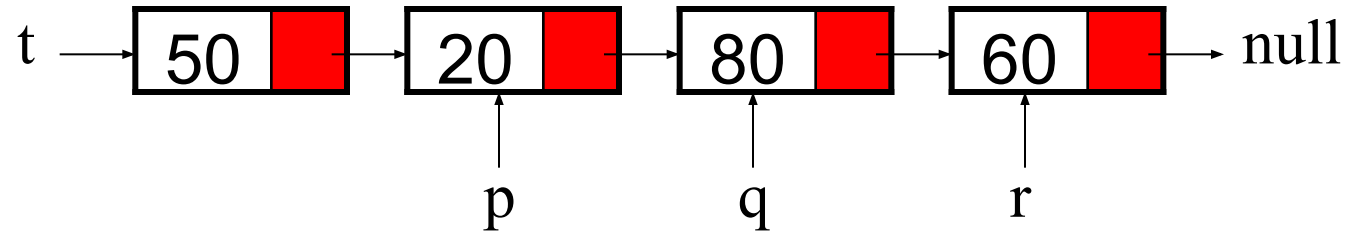
  - x.Data: access to the "data" of node x
  - x.Link: access to the "link" of node x

- If node have no name, but pointed by pointer t:



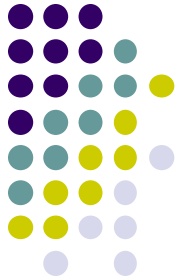
  - t→Data: access to the "data" of node pointed by t
  - t → Link: access to the "link" of node pointed by t

- Ex:



  - Print(p→Data) = 20
  - Print((p→Link)→) = 80
  - r is equivalent to q→Link, (p→link)→link

Linked List: Properties

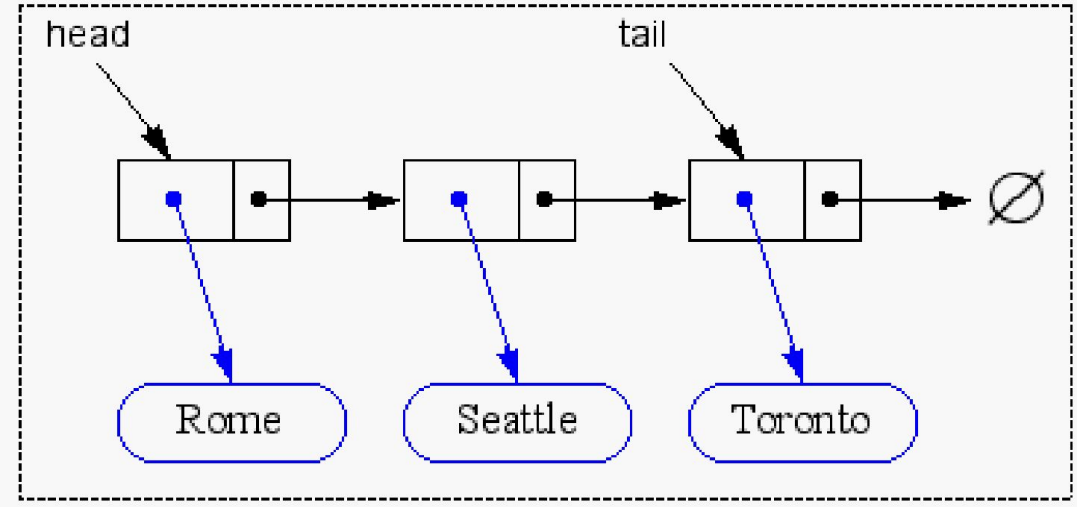


- Properties of Linked list:
  - **Nodes may not stored in serial Memory**
    - One of the attributes of a linked list is that **there is not a physical relationship** between the nodes; that is, they are not stored contiguously.
  - **Data Type of different Nodes can be different**
  - **Easy to Insert/Delete Node**

    It is not necessary to shift elements of a linked list to make room for a new element or to delete an element.

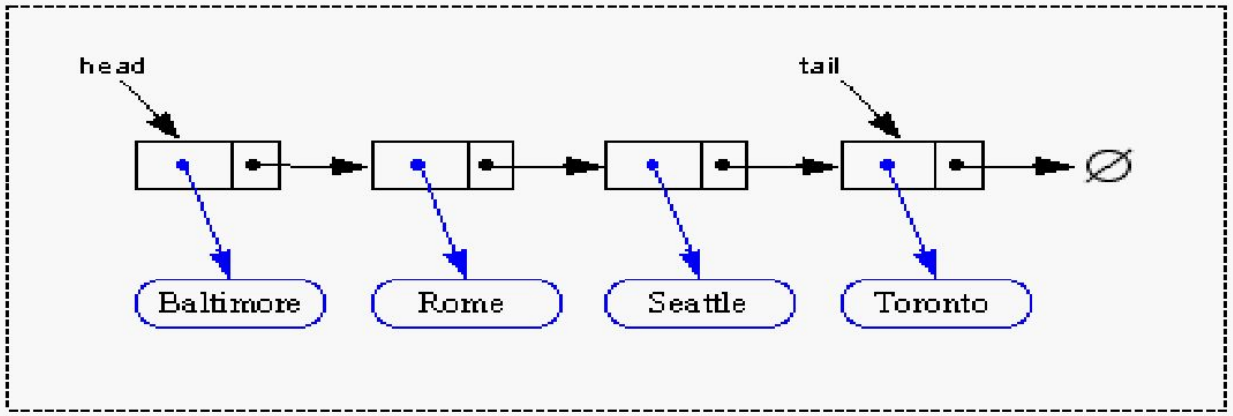
Implementing a Queue with a Singly Linked List

Nodes connected in a chain by links

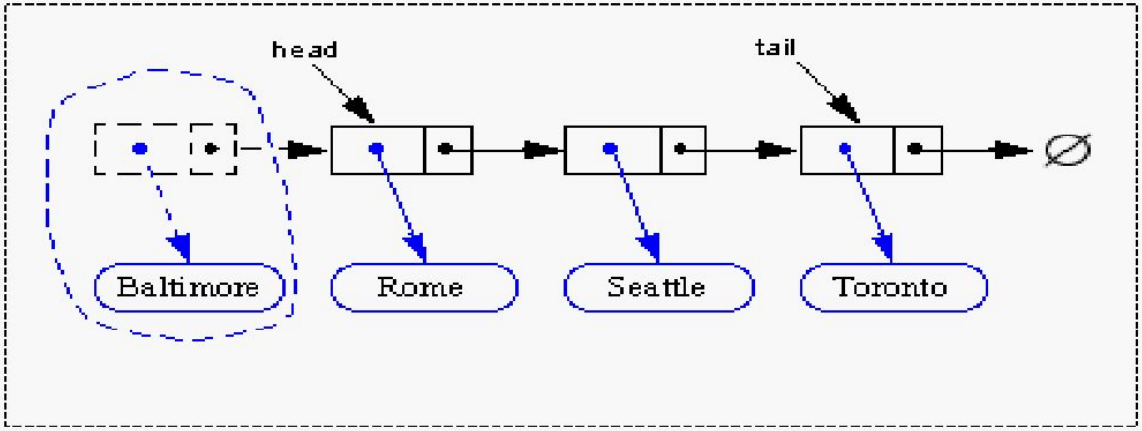


The head of the list is the front of the queue, the tail of the list is the rear of the queue. ***Why not the opposite?***

Removing at the Head



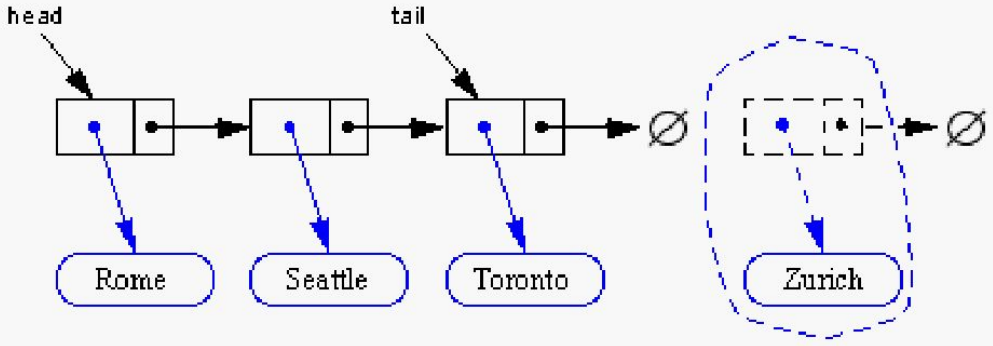
- advance head reference



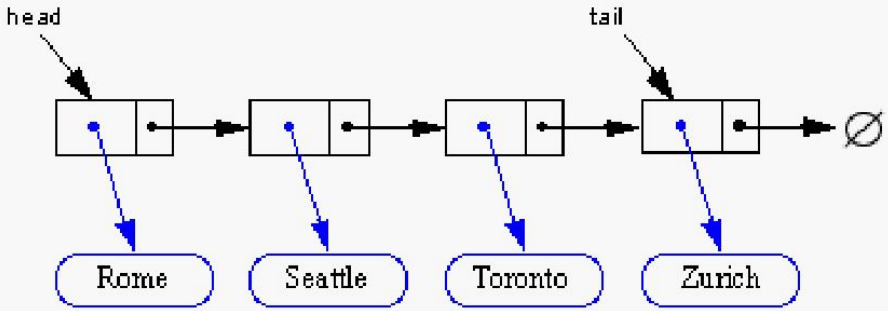
- inserting at the head is just as easy

Inserting at the Tail

- create a new node

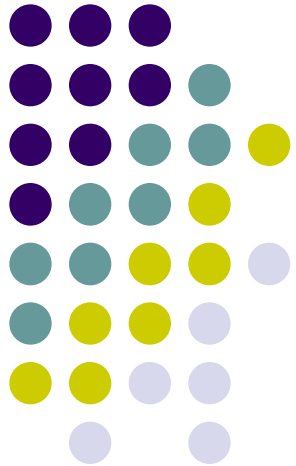


- chain it and move the tail reference

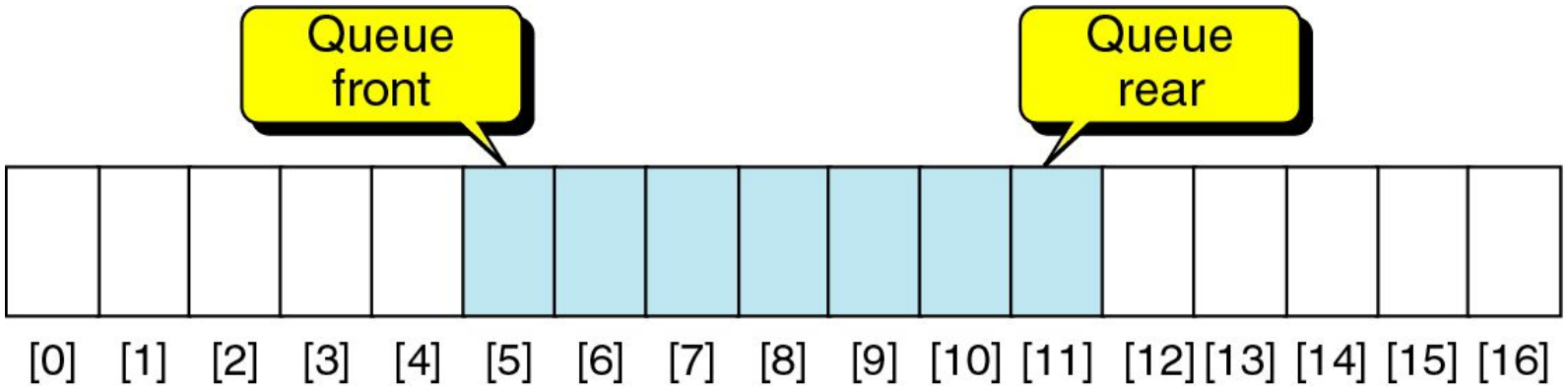


- how about removing at the tail?

Circular Queue & Linked List



Circular Queue



- Use Linear Array to implement a queue.
- Waste of memory: The deleted elements can not be re-used.
- Solution: to use circular queue.
- Two implementations:
  - Using n-1 space.
  - Using n space + full tag

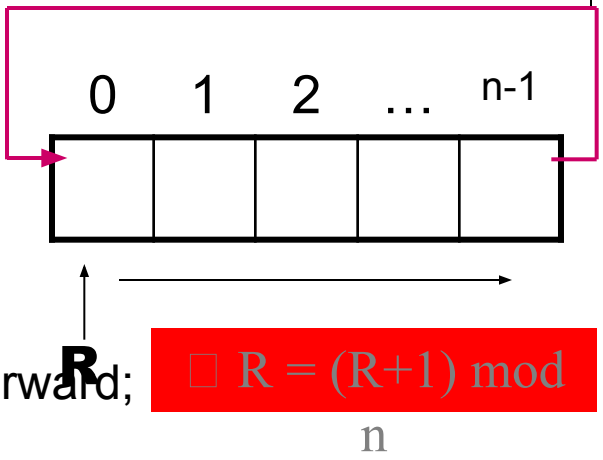
Implementation of Circular queue with (n-1) space used

- **Create(Q)**

  Q: Array[0…n-1]

  front = rear = 0 //initialize

- **Enqueue(item, Q) Queue**



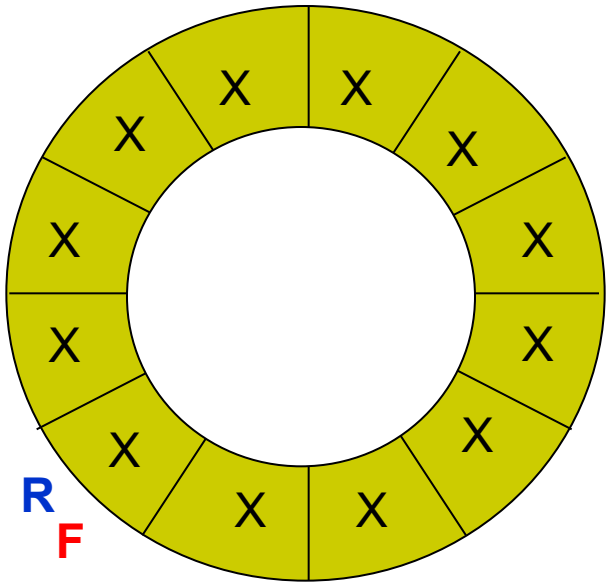

```
begin
    rear = (rear+1) mod n;    //rear moves forward;
    if rear = front
        QueueFull;      // Queue is full.
        rear = rear-1 mod n;    //  rear back to the previous position;
    else
        Q[rear]=item;
end;
```

Implementation of Circular queue with (n-1) space used

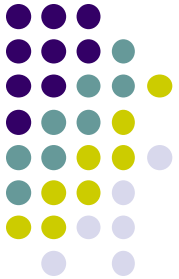
- Dequeue(Q) □item

```
begin
    if front=rear
        QueueEmpty;
    else
        front = (front+1) mod n;
        item = Q[front];
    end;
end;
```

- Note: only **(n-1)** space used;

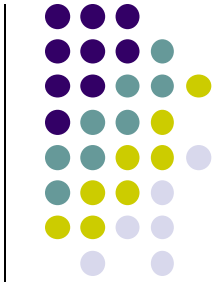


Implementation of Circular Queue with n space used



- A parameter "Tag" is introduced to help to make sure the queue is Empty or Full:
  - Boolean
  - If Tag = True, combined with other conditions => queue is Full
  - If Tag = False, combined with other conditions => queue is Null
  - **"Tag" can determine the states of the queue solely!**

Implementation of Circular Queue with n space used



- **Create(Q)**

```
Q: Array[0…n-1]
front = rear: int = 0
Tag: Boolean = False
```

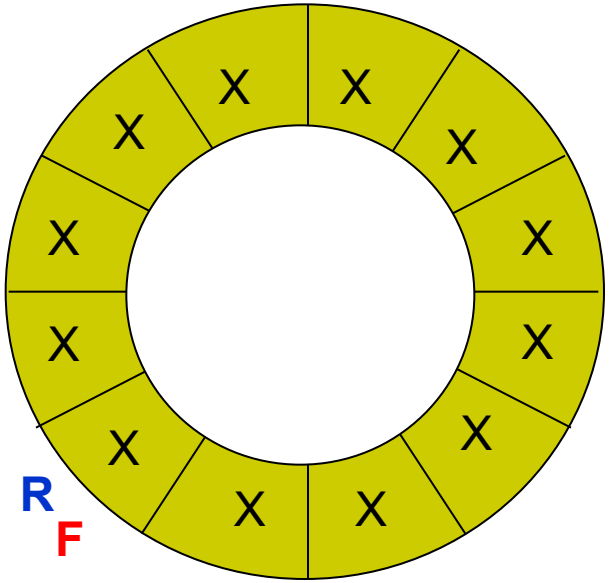
- **Enqueue(item, Q) Queue**

```
begin
 if (rear = front &Tag = 1)
     QueueFull;
 else begin
          rear = (rear+1) mod n;    //rear moves forward;
          Q[rear]=item;
          if (rear=front)
               Tag=1;
       end;
 end;
```

Implementation of Circular Queue with n space used

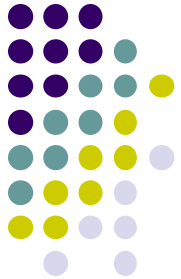
- **Dequeue(Q) item**

```
begin
    if (Front=Rear and Tag=0)
        QueueEmpty;
    else begin
            Front = (Front+1) mod n;
            item = Q[Front];
            if (Front=Rear)
                Tag=0;
          end;
end;
```



- **Note:** n space can be used

Double-Ended Queues



A double-ended queue, or deque, supports insertion and deletion from the front and back.

The Deque Abstract Data Type

insertFirst(e):	Insert e at the deginning of deque.
insertLast(e):	Insert e at end of deque
removeFirst():	Removes and returns first element
removeLast():	Removes and returns last element

Additionally supported methods include:

- first()
- last()
- size()
- isEmpty()

Implementing Stacks and Queues with Deques



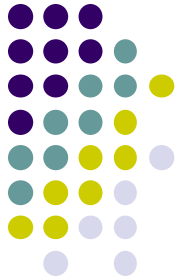
Stacks with Deques:

Stack Method	Deque Implementation
size()	size()
isEmpty()	isEmpty()
top()	last()
push(e)	insertLast(e)
pop()	removeLast()

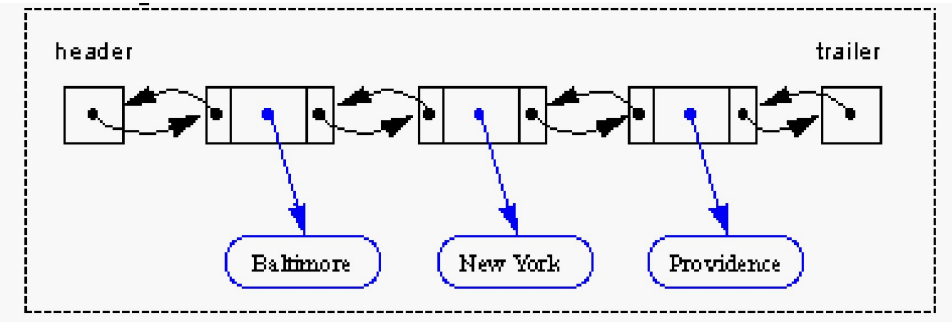
Queues with Deques:

Queue Method	Deque Implementation
size()	size()
isEmpty()	isEmpty()
front()	first()
enqueue()	insertLast(e)
dequeue()	removeFirst()

Implementing Deques with Doubly Linked Lists



- Deletions at the tail of a singly linked list cannot be done in constant time.
- To implement a deque, we use a doubly linked list. with special header and trailer nodes



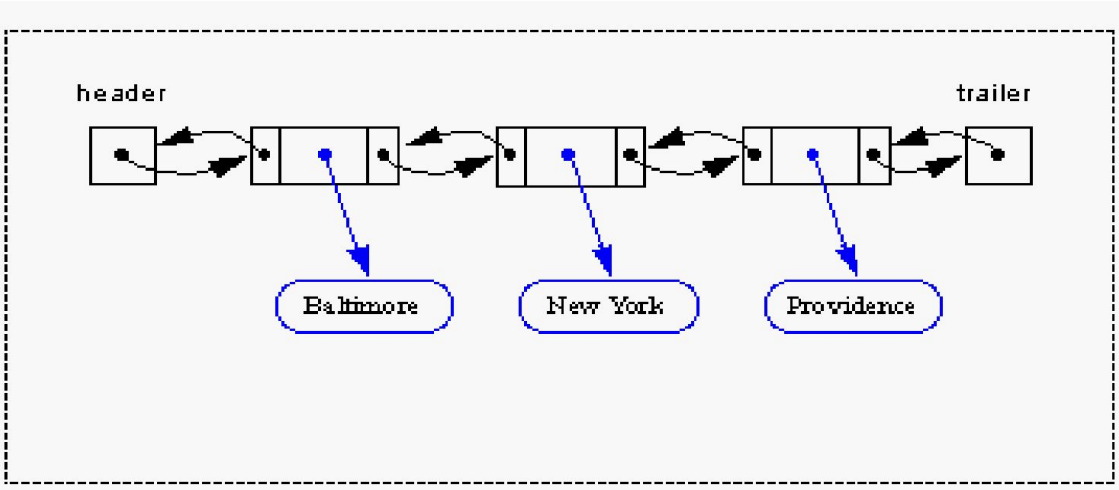
- A node of a doubly linked list has a next and a prev link. It supports the following methods:
  - setElement(Object e)
  - setNext(Object newNext)
  - setPrev(Object newPrev)
  - getElement()
  - getNext()
  - getPrev()
- By using a doubly linked list, all the methods of a deque run in O(1) time.

Implementing Deques with Doubly Linked Lists (cont.)

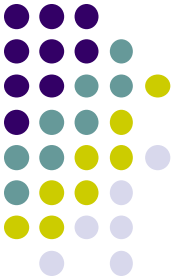
When implementing a doubly linked lists, we add two special nodes to the ends of the lists: the header and trailer nodes.

- The ***header*** node goes before the first list element. It has a valid next link but a null prev link.
- The ***trailer*** node goes after the last element. It has a valid prev reference but a null next reference.

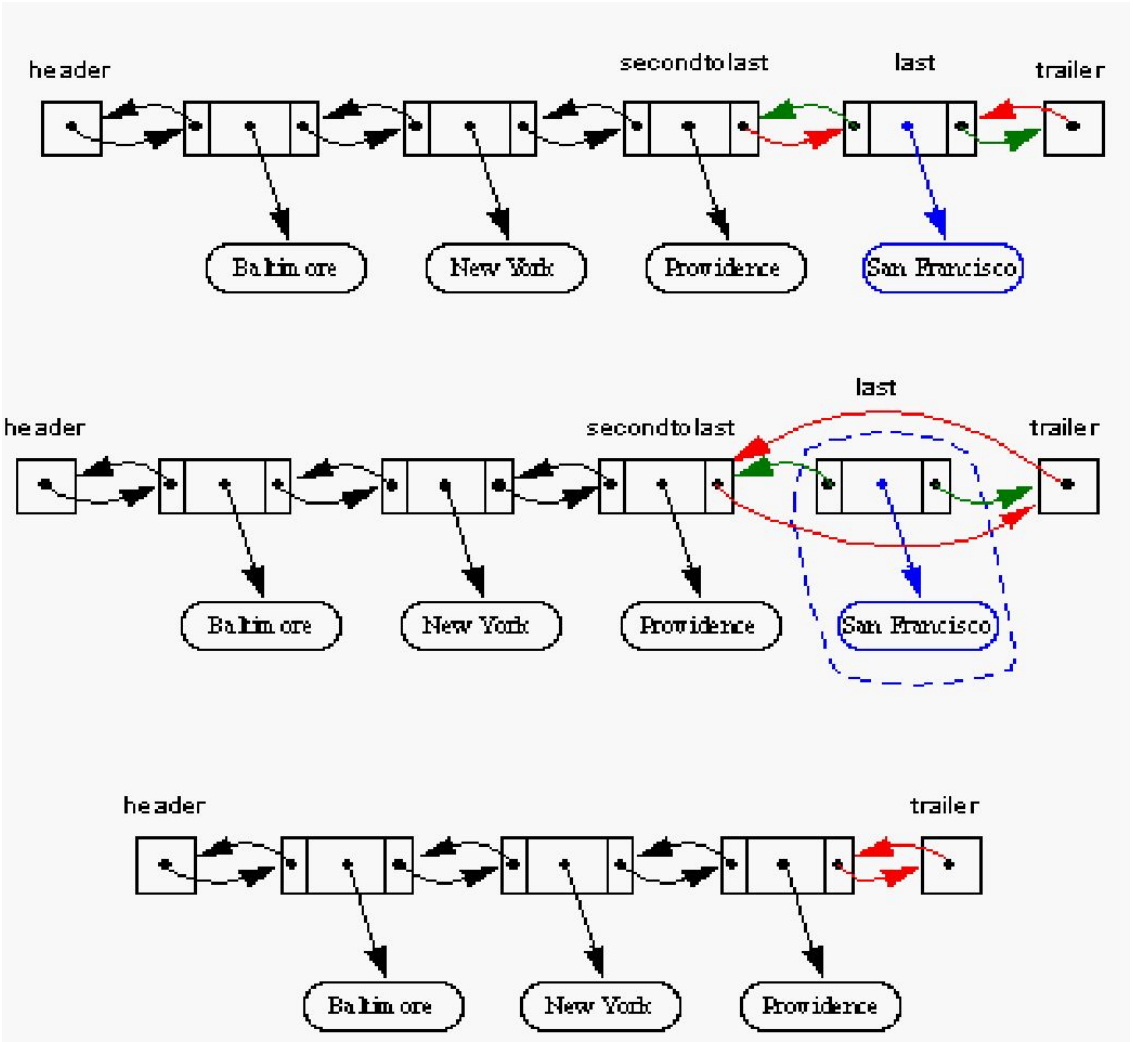
NOTE: ***the header and trailer nodes are sentinel or "dummy" nodes because they do not store elements. Here's a diagram of our doubly linked list:***



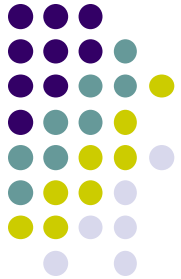
Implementing Deques with Doubly Linked Lists (cont.)



Here's a visualization of the code for removeLast().

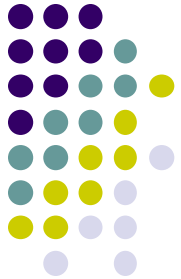


Priority Queue



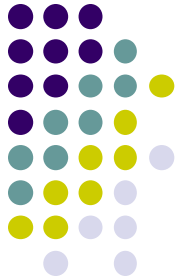
- Only one check-in service in Air Canada at airport
- Two waiting lines for passengers
  - one is First class service
  - the other is Economy class service
- Passengers in the first-class waiting line have higher priority to check in than those in the economy-class waiting line.

Priority Queue



- Two queues
  - one is high priority queue
  - the other is low priority queue
- Service rules:
  - First serve the people in high priority queue
  - If no passengers are in high priority queue, serve the passengers in low priority queue

The Priority Queue ADT



- A priority queue P supports the following methods:

-size(): Return the number of elements in P

-isEmpty(): Test whether P is empty

-insertItem(k,e): Insert a new element e with key k into P

-minElement(): Return (but don't remove) an element of P with smallest key; an error occurs if P is empty.

-minKey(): Return the smallest key in P; an error occurs if P is empty

-removeMin(): Remove from P and return an element with the smallest key; an error condidtion occurs if P is empty.

Two Queues

- High Priority Queue, will come in *hpQue*
- Low Priority Queue, will come in *lpQue*

